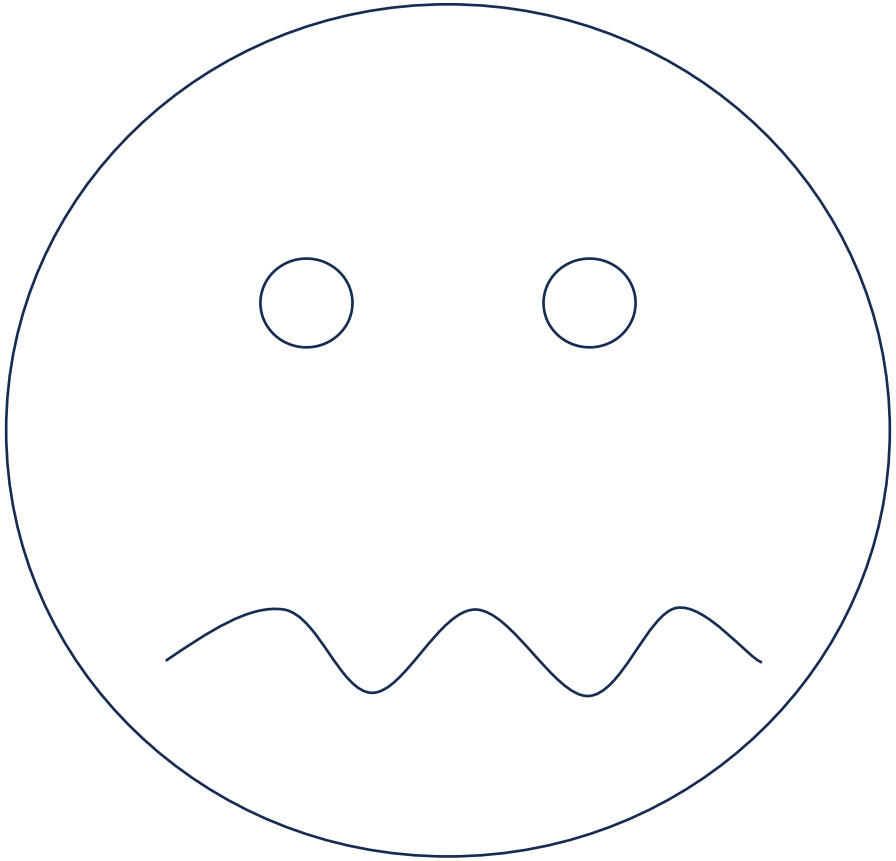
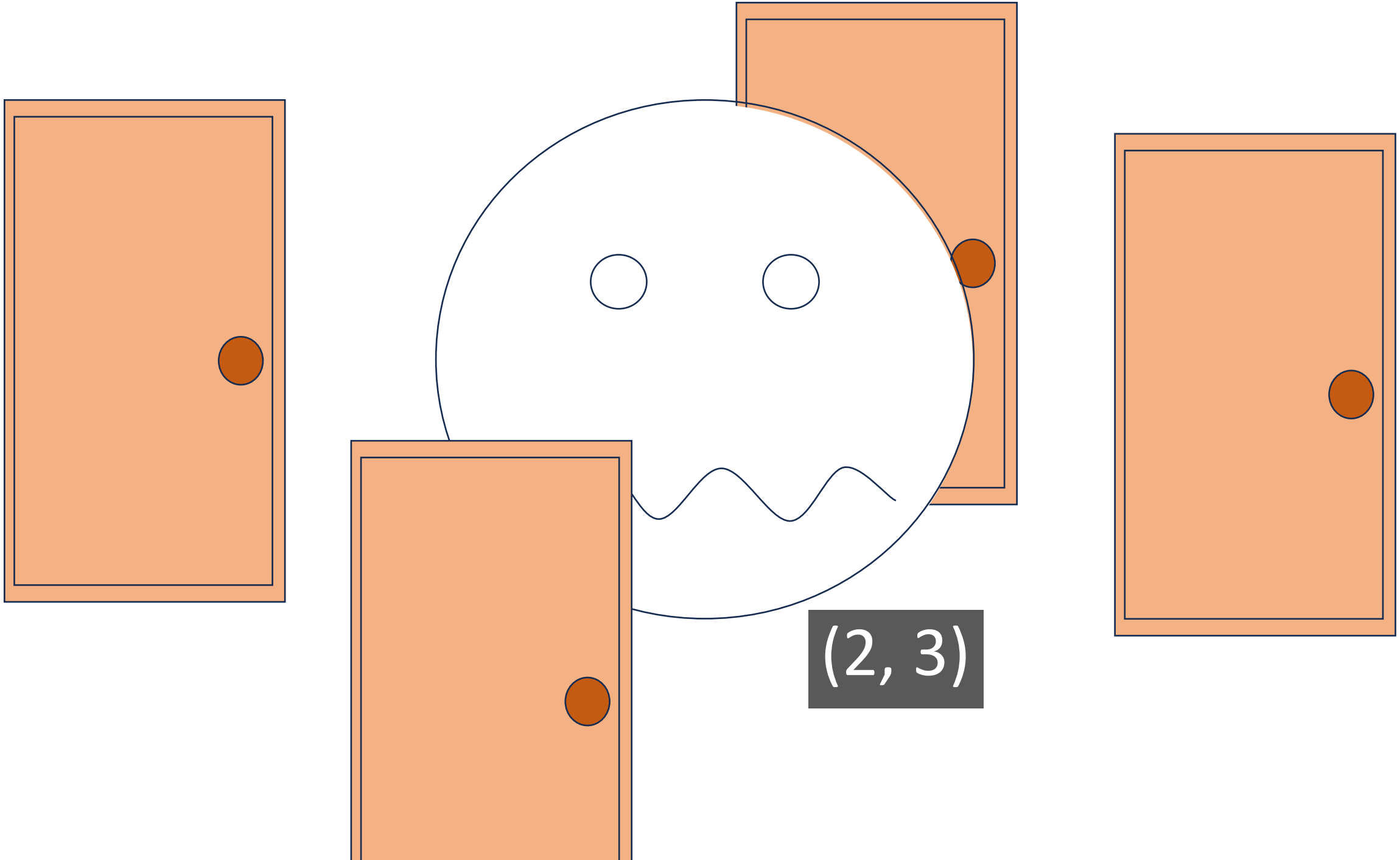
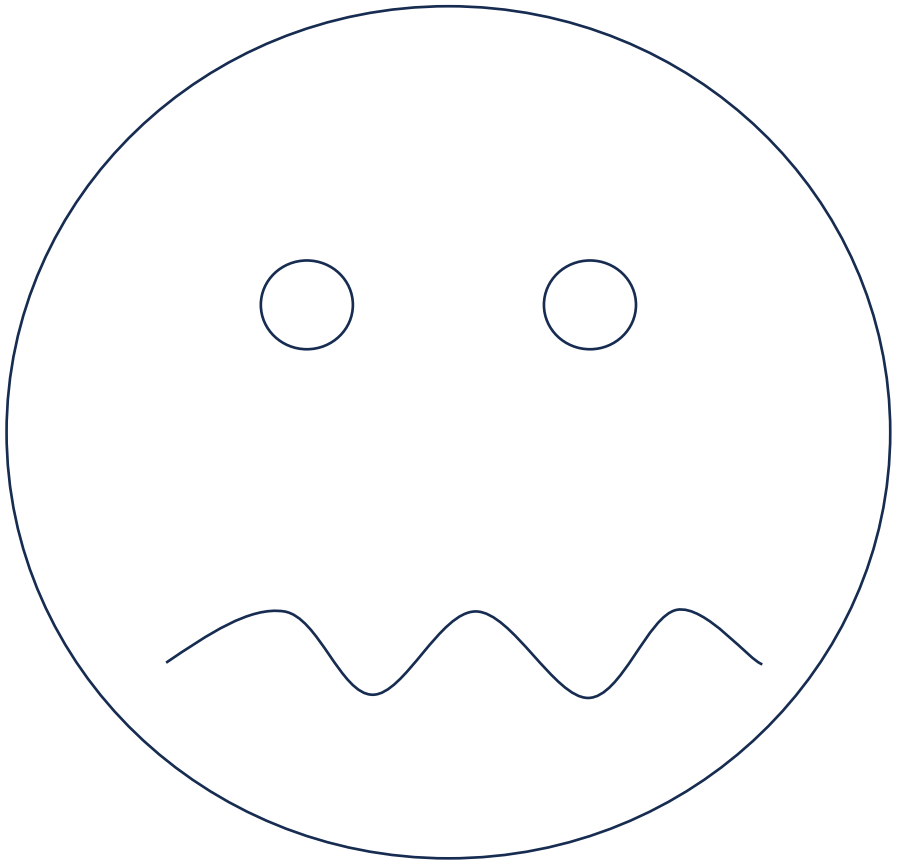
Exercise 8.4 (programming)

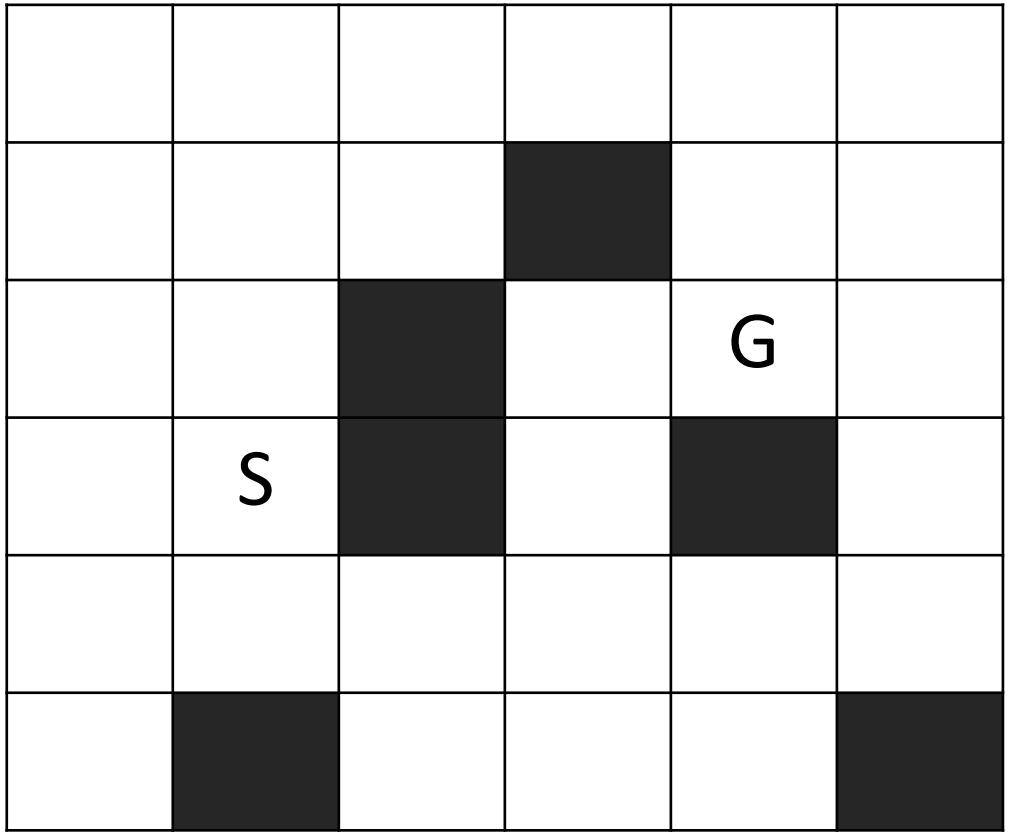
The exploration bonus described above actually changes the estimated values of states and actions. Is this necessary? Suppose the bonus $\kappa\sqrt{\tau}$ was used not in updates, but solely in action selection. That is, suppose the action selected was always that for which $Q(S_t, a) + \kappa\sqrt{\tau(S_t, a)}$ was maximal. Carry out a gridworld experiment that tests and illustrates the strengths and weaknesses of this alternate approach.



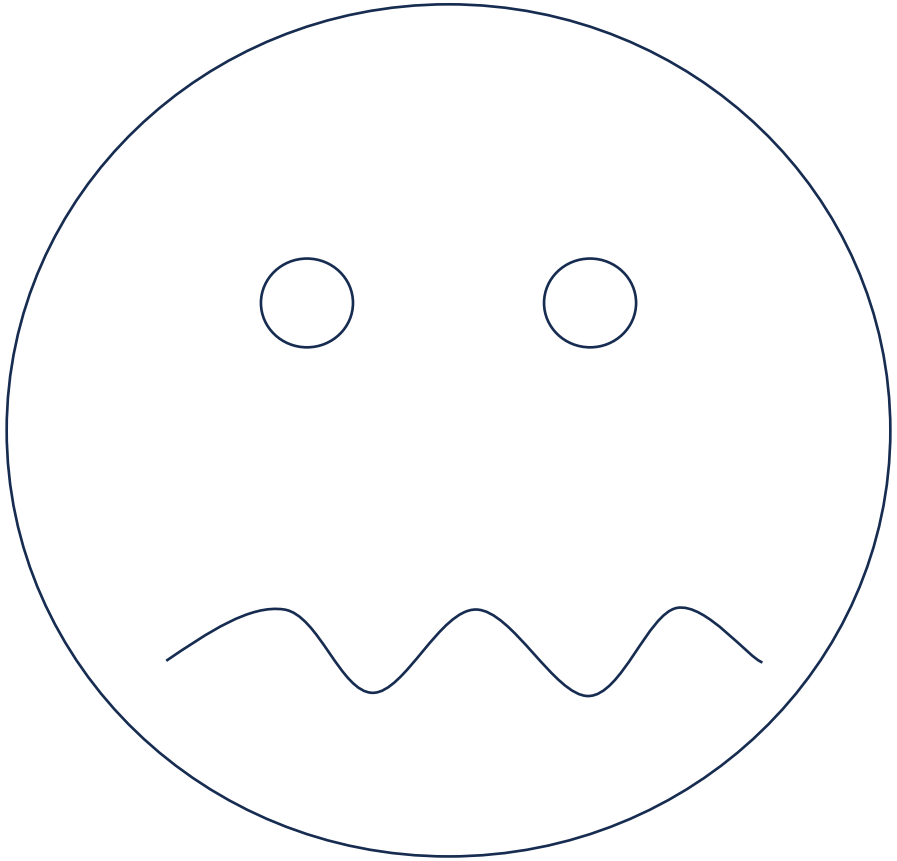
This is a Bob.


(2, 3)




G
S

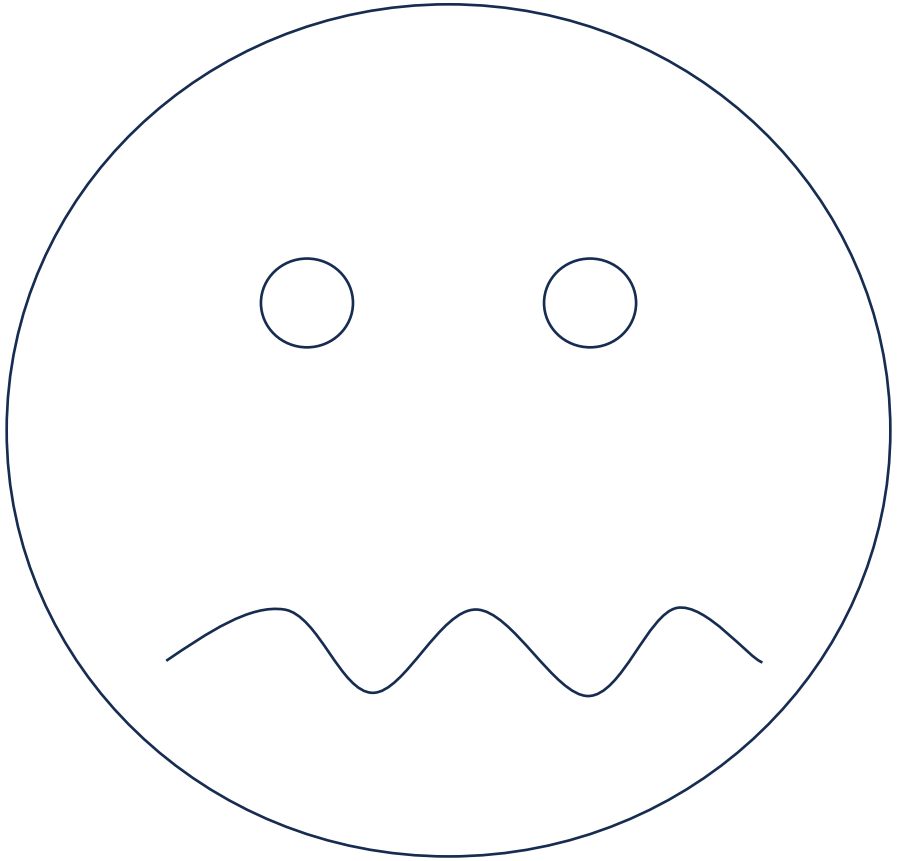
Blind



He wants to get out of this maze as soon as possible!

Has a compass

Blind

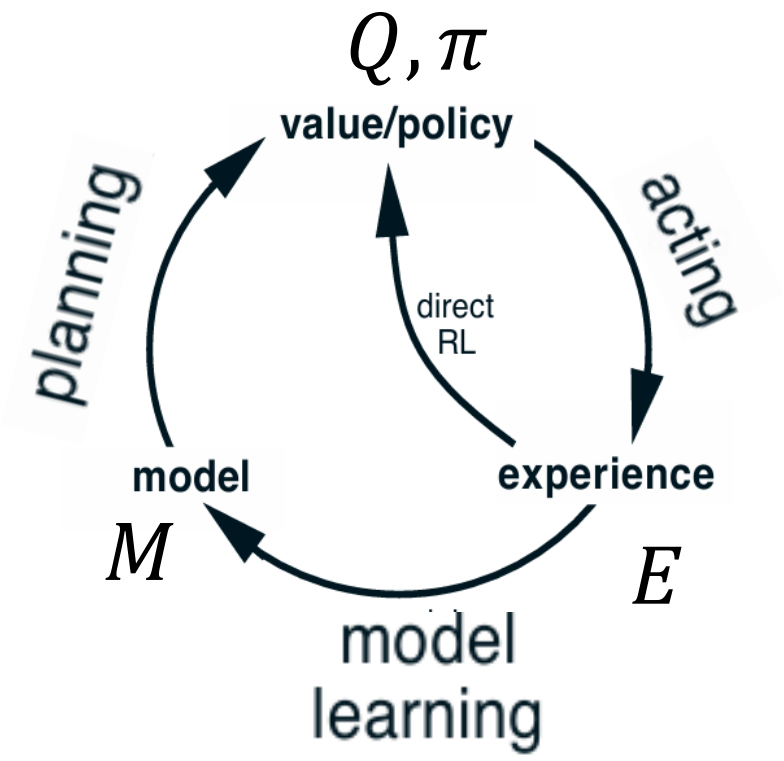


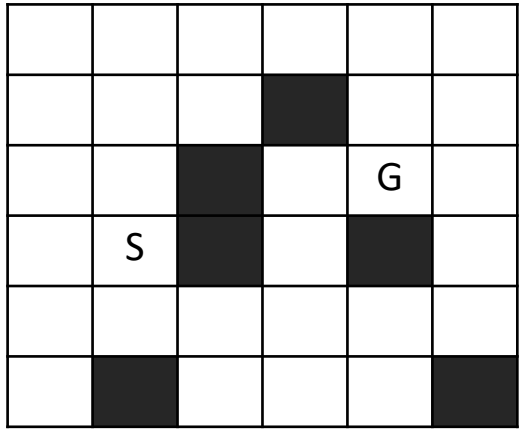
Perfect Memo.

High Ambition

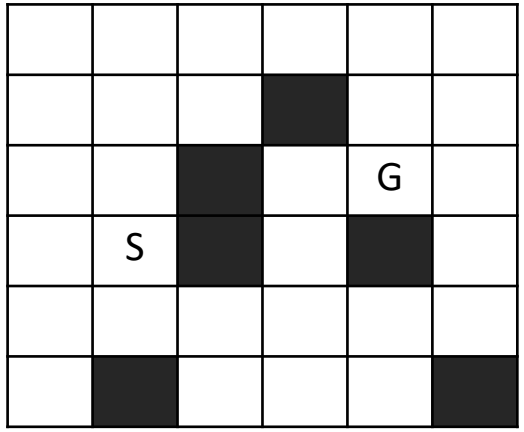
He wants to get out of this maze as soon as possible!

Dyna-Q

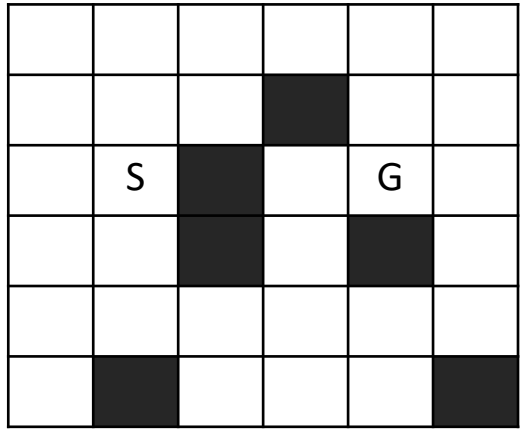

Q, π
value/policy
planning
acting
direct
RL
model
M
experience
E
model
learning



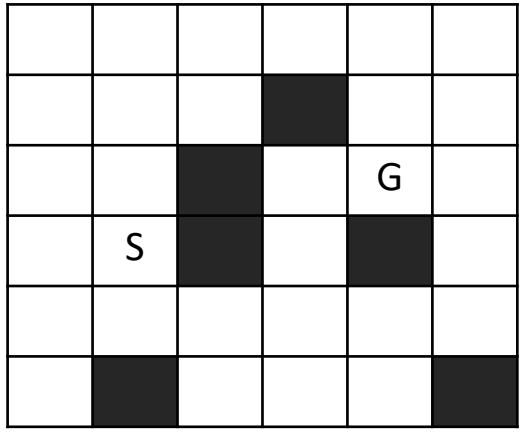
No Information



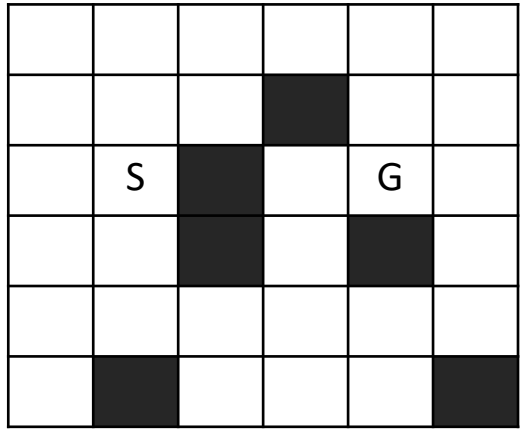
No Information



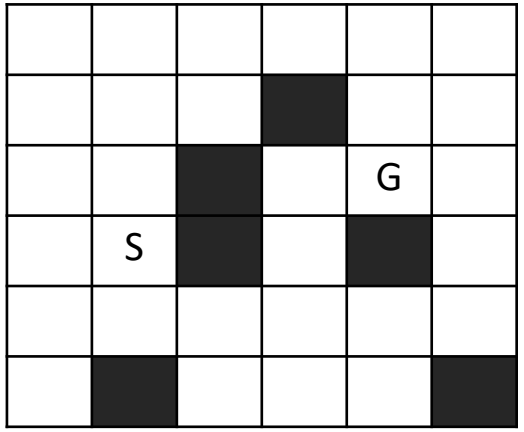
Up, no exit!



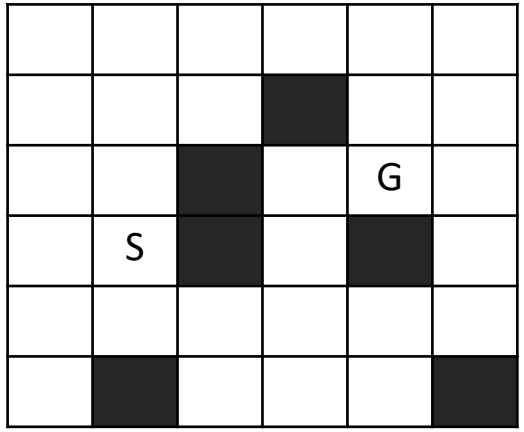
No Information



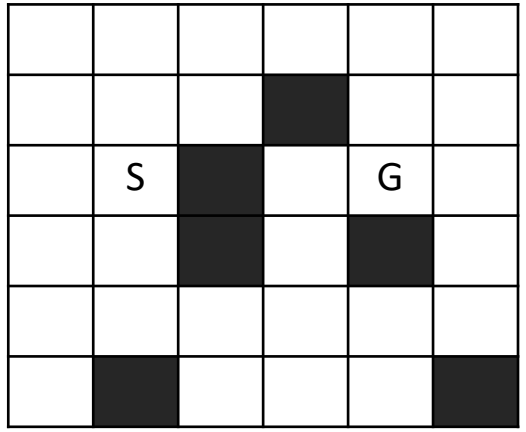
Up, no exit!



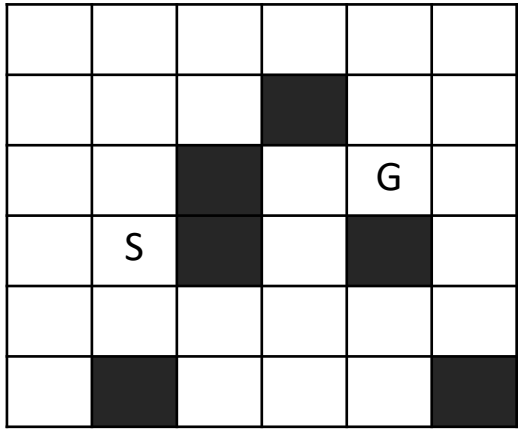
Down, no exit!



No Information

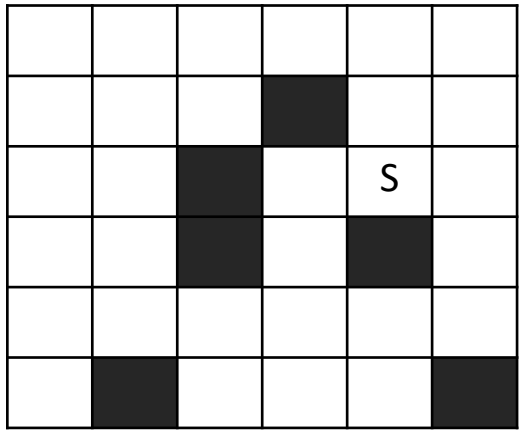


Up, no exit!

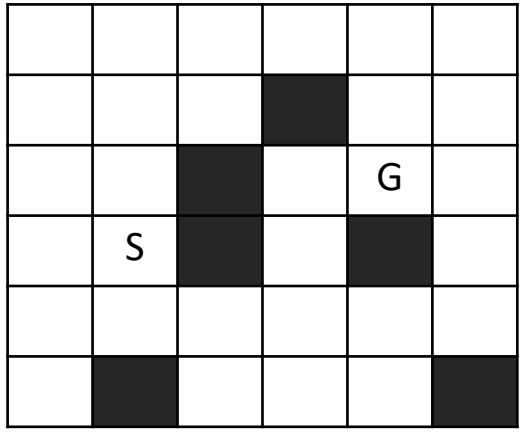


Down, no exit!

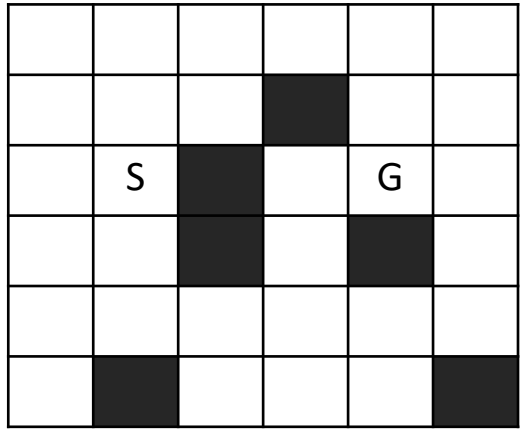
.....



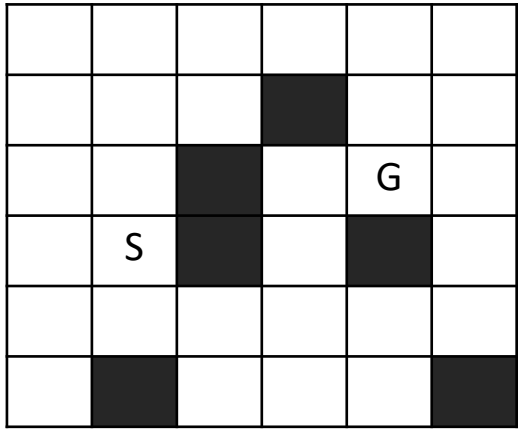
Yay, Exit!



No Information

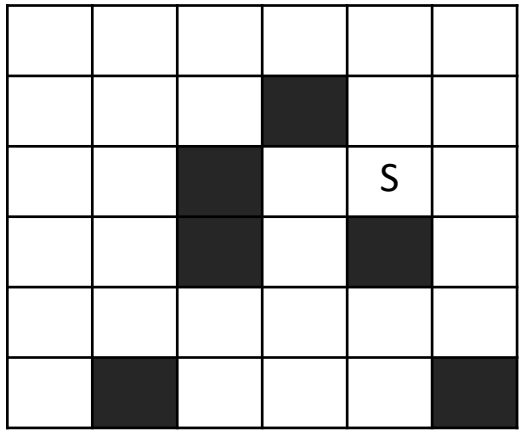


Up, no exit!



Down, no exit!

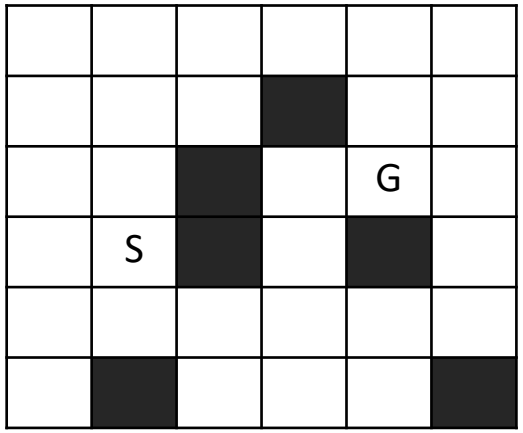
.....



Yay, Exit!

Send back

To the start



Dyna-Q

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Required: $\mathrm{n}, \alpha, \epsilon, \gamma$

Dyna-Q

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:

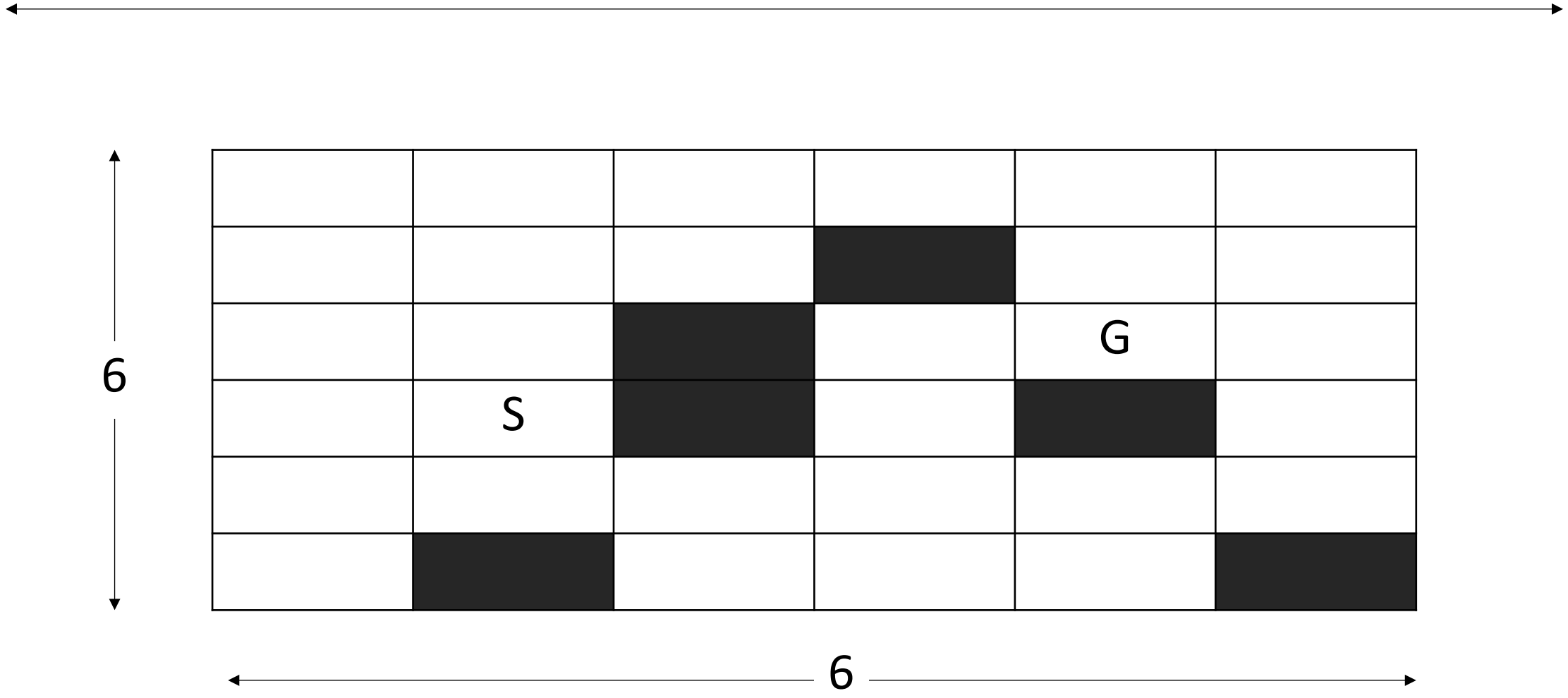
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:
$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Required: $\mathrm{n}, \alpha, \epsilon, \gamma$

Real Interaction Phase

Planning Phase

Setup the problem



GridWorld

```python
class GridWorld:
    """ A grid world environment (width x height) for reinforcement learning.
    Each cell in the grid can be occupied by an agent, a goal, or an obstacle.


    Args:
        width (int): The width of the grid world.
        height (int): The height of the grid world.

    """
    def __init__(self, width: int, height: int):
        self.width = width
        self.height = height
        self.size = (width,  height)
        self.obstacles = set()
        self.goals = defaultdict(float)  # {(x, y): reward}
```

```python
World = GridWorld(10, 10)
World.display()
```

✓ 0.0s

```
~~~~~~~~~ Grid World ~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    .    .  |
 9 |  .    .    .    .    .    .    .    .    .    .  |
 8 |  .    .    .    .    .    .    .    .    .    .  |
 7 |  .    .    .    .    .    .    .    .    .    .  |
 6 |  .    .    .    .    .    .    .    .    .    .  |
 5 |  .    .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
    ------------------------------------------------
      1    2    3    4    5    6    7    8    9   10
```

GridWorld

```python
def add_obstacle(self, x: int, y: int):
    """ Add an obstacle at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        self.obstacles.add((x, y))
    else:
        raise ValueError("Coordinates are out of bounds.")

def add_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Add multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.add_obstacle(x, y)

def remove_obstacle(self, x: int, y: int):
    """ Remove an obstacle from the specified coordinates (x, y). """
    self.obstacles.discard((x, y))

def remove_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Remove multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.remove_obstacle(x, y)
```

GridWorld

```python
def add_obstacle(self, x: int, y: int):
    """ Add an obstacle at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        self.obstacles.add((x, y))
    else:
        raise ValueError("Coordinates are out of bounds.")

def add_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Add multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.add_obstacle(x, y)

def remove_obstacle(self, x: int, y: int):
    """ Remove an obstacle from the specified coordinates (x, y). """
    self.obstacles.discard((x, y))

def remove_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Remove multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.remove_obstacle(x, y)
```

```python
World.add_obstacle(2,2)
```

```
~~~~~~~~~ Grid World ~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    .    .  |
 9 |  .    .    .    .    .    .    .    .    .    .  |
 8 |  .    .    .    .    .    .    .    .    .    .  |
 7 |  .    .    .    .    .    .    .    .    .    .  |
 6 |  .    .    .    .    .    .    .    .    .    .  |
 5 |  .    .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .    .  |
 2 |  .    #    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
    ---------------------------------------------------
      1    2    3    4    5    6    7    8    9   10
```

GridWorld

```python
def add_obstacle(self, x: int, y: int):
    """ Add an obstacle at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        self.obstacles.add((x, y))
    else:
        raise ValueError("Coordinates are out of bounds.")

def add_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Add multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.add_obstacle(x, y)

def remove_obstacle(self, x: int, y: int):
    """ Remove an obstacle from the specified coordinates (x, y). """
    self.obstacles.discard((x, y))

def remove_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Remove multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.remove_obstacle(x, y)
```

```python
World.add_obstacles([(2, 2), (3, 3), (4, 4),
                     (5, 5), (6, 6), (7, 7),
                     (8, 8), (9, 9), (9, 10)])
```

```
~~~~~~~~~ Grid World ~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    .    .  |
 9 |  .    .    .    .    .    .    .    .    .    .  |
 8 |  .    .    .    .    .    .    .    .    .    .  |
 7 |  .    .    .    .    .    .    .    .    .    .  |
 6 |  .    .    .    .    .    .    .    .    .    .  |
 5 |  .    .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
   ---------------------------------------------------
      1    2    3    4    5    6    7    8    9   10
```

GridWorld

```python
def add_obstacle(self, x: int, y: int):
    """ Add an obstacle at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        self.obstacles.add((x, y))
    else:
        raise ValueError("Coordinates are out of bounds.")

def add_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Add multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.add_obstacle(x, y)

def remove_obstacle(self, x: int, y: int):
    """ Remove an obstacle from the specified coordinates (x, y). """
    self.obstacles.discard((x, y))

def remove_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Remove multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.remove_obstacle(x, y)
```

```python
World.remove_obstacle(4,4)
```

```
~~~~~~~~~ Grid World ~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    ▮    .  |
 9 |  .    .    .    .    .    .    .    .    ▮    .  |
 8 |  .    .    .    .    .    .    .    ▮    .    .  |
 7 |  .    .    .    .    .    .    ▮    .    .    .  |
 6 |  .    .    .    .    .    ▮    .    .    .    .  |
 5 |  .    .    .    .    ▮    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    ▮    .    .    .    .    .    .    .  |
 2 |  .    ▮    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
    ---------------------------------------------------
      1    2    3    4    5    6    7    8    9   10
```

GridWorld

```python
def add_obstacle(self, x: int, y: int):
    """ Add an obstacle at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        self.obstacles.add((x, y))
    else:
        raise ValueError("Coordinates are out of bounds.")

def add_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Add multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.add_obstacle(x, y)

def remove_obstacle(self, x: int, y: int):
    """ Remove an obstacle from the specified coordinates (x, y). """
    self.obstacles.discard((x, y))

def remove_obstacles(self, obstacle_list: list[tuple[int, int]]):
    """ Remove multiple obstacles from a list of coordinates. """
    for x, y in obstacle_list:
        self.remove_obstacle(x, y)
```

```python
World.remove_obstacles([(4, 4), (6,6), (7,7)])
```

```
~~~~~~~~~ Grid World ~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    .    .  |
 9 |  .    .    .    .    .    .    .    .    .    .  |
 8 |  .    .    .    .    .    .    .    .    .    .  |
 7 |  .    .    .    .    .    .    .    .    .    .  |
 6 |  .    .    .    .    .    .    .    .    .    .  |
 5 |  .    .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
    ---------------------------------------------------
      1    2    3    4    5    6    7    8    9   10
```

GridWorld

```python
def add_goal(self, x: int, y: int, reward: float = 1.0):
    """ Add a goal at the specified coordinates (x, y). """
    if 1 <= x <= self.width and 1 <= y <= self.height:
        if (x, y) in self.obstacles:
            raise ValueError("Cannot place a goal on an obstacle.")

        self.goals[(x, y)] = reward

    else:
        raise ValueError("Coordinates are out of bounds.")


def add_goals(self, goal_list: list[tuple[int, int, float]]):
    """ Add multiple goals from a list of coordinates. """
    for x, y, reward in goal_list:
        self.add_goal(x, y, reward)

def remove_goal(self, x: int, y: int):
    """ Remove a goal from the specified coordinates (x, y). """
    for goal in list(self.goals):
        if goal[0] == x and goal[1] == y:
            self.goals.discard(goal)
            break

def remove_goals(self, goal_list: list[tuple[int, int]]):
    """ Remove multiple goals from a list of coordinates. """
    for x, y in goal_list:
        self.remove_goal(x, y)
```

GridWorld

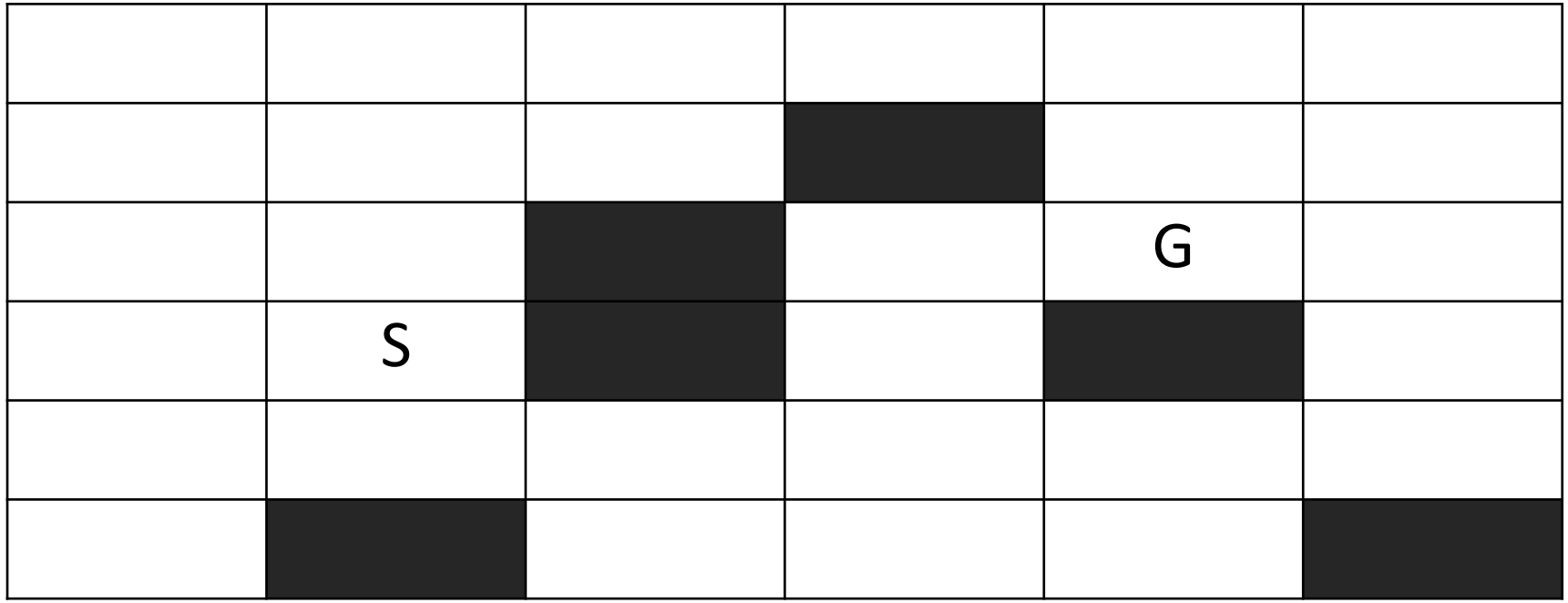
```python
World = GridWorld(10, 10)
World.add_goal(10, 10, reward=10.0)
World.add_obstacles([(2, 2), (3, 3), (4, 4),
                     (5, 5), (6, 6), (7, 7),
                     (8, 8), (9, 9), (9, 10)])
World.display()
```

```
~~~~~~~~~~ Grid World ~~~~~~~~~~
10 |  .    .    .    .    .    .    .    .    .    G  |
 9 |  .    .    .    .    .    .    .    .    .    .  |
 8 |  .    .    .    .    .    .    .    .    .    .  |
 7 |  .    .    .    .    .    .    .    .    .    .  |
 6 |  .    .    .    .    .    .    .    .    .    .  |
 5 |  .    .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .    .  |
    ---------------------------------------------------
      1    2    3    4    5    6    7    8    9   10

--- Rewards ---
Goal at (10, 10): 10.00
```

Dyna-Q

```python
class Dyna_Q_Refined:
    """
    Dyna-Q algorithm working with a GridWorld environment.

    Args:
        world (GridWorld): The environment in which the agent will learn and plan.
    """
    def __init__(self, world):
        self.world = world
        self.width, self.height = world.size
        self.obstacles = world.obstacles
        self.goals = world.goals
        self.available_states = {(x, y) for x in range(1, self.width + 1)
                                 for y in range(1, self.height + 1)} - self.obstacles
        self.time_step = 0
        self.last_visit = defaultdict(int)
```

Dyna-Q



Dyna-Q

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$-greedy(S,Q)

(c) Take action A; observe resultant reward, R, and state, S'

(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S,A)$

$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$

(b) $A \leftarrow$

(c) Take

(d) $Q(S,$

(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop r

$S \leftarrow$

$A \leftarrow$

R, S'

$Q(S,$

```python
def train(self, num_episode, num_plan, alpha=0.1, epsilon=0.1, gamma=0.9, k=0.1, seed=None,
          fixed_start_point=None, max_steps=10000, set_zero=False):

    self.initialize(seed=seed, set_zero=set_zero, alpha=alpha, epsilon=epsilon,
                    gamma=gamma, k=k, fixed_start_point=fixed_start_point, max_steps=max_steps)
```

```python
def initialize(self, seed=None, set_zero=False, alpha=0.1, epsilon=0.1,
               gamma=0.9, k=0.1, fixed_start_point=None, max_steps=10000):
    if seed is not None: random.seed(seed)
    self.Q = defaultdict(lambda: {a: 0.0 if set_zero else random.gauss(0, 1)
                                  for a in ['up', 'down', 'left', 'right']})
    self.model = {}

    self.alpha = alpha
    self.gamma = gamma
    self.k = k
    self.epsilon = epsilon
    self.fixed_start_point = fixed_start_point
    self.max_steps = max_steps
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:

(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:

$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$-greedy(S,Q)

(c) Take action A; observe resultant reward, R, and state, S'

(d) Q

(e) M

(f) L

```python
        self.initialize(seed=seed, set_zero=set_zero, alpha=alpha, epsilon=epsilon,
                        gamma=gamma, k=k, fixed_start_point=fixed_start_point, max_steps=max_ste

        for _ in tqdm(range(num_episode)):
            if self.fixed_start_point is not None:
                state = self.fixed_start_point
            else:
                state = random.choice(list(self.available_states))


            for step in range(self.max_steps):  # Limit episode length
                self.time_step += 1

                # Choose action
                action = self.get_action(state, epsilon, k)

                # Take action and observe next state and reward
                next_state, reward = self.take_action(state, action)
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:

(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:
$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

```python
def get_action(self, state, epsilon, k=0):
    if random.random() < epsilon:
        return random.choice(['up', 'down', 'left', 'right'])
    # Default behavior: standard Q-max (k=0)
    return max(self.Q[state], key=lambda a: self.Q[state][a])
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

```python
for step in range(self.max_steps):  # Limit episode length
    # Choose action
    action = self.get_action(state, epsilon, k)

    # Take action and observe next state and reward
    next_state, reward = self.take_action(state, action)

    self.last_visit[(state, action)] = self.time_step

    # Direct RL Update
    self.update_q(state, action, reward, next_state, alpha, gamma)
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$-greedy(S,Q)

(c) Take action A; observe resultant reward, R, and state, S'

(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$

```python
def take_action(self, state, action):
    x, y = state
    move = {'up': (0, 1), 'down': (0, -1), 'left': (-1, 0), 'right': (1, 0)}
    dx, dy = move[action]
    next_state = (x + dx, y + dy)

    # Boundary and obstacle check
    if (next_state in self.obstacles) or not (1 <= next_state[0] <= self.width) or \
       not (1 <= next_state[1] <= self.height):
        next_state = state

    return next_state, self.goals.get(next_state, 0.0)
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

```python
# Take action and observe next state and reward
next_state, reward = self.take_action(state, action)

self.last_visit[(state, action)] = self.time_step

# Direct RL Update
self.update_q(state, action, reward, next_state, alpha, gamma)

# Model Update
if state not in self.model: self.model[state] = {}
self.model[state][action] = (reward, next_state)
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

```python
def update_q(self, s, a, r, next_s, alpha, gamma):
    best_q = max(self.Q[next_s].values())
    self.Q[s][a] += alpha * (r + gamma * best_q - self.Q[s][a])
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:

(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:
$S \leftarrow$ random previously observed state

```python
# Direct RL Update
self.update_q(state, action, reward, next_state, alpha, gamma)

# Model Update
if state not in self.model: self.model[state] = {}
self.model[state][action] = (reward, next_state)

# Planning
for _ in range(num_plan):
    s_sim = random.choice(list(self.model.keys()))
    a_sim = random.choice(list(self.model[s_sim].keys()))
    r_sim, next_s_sim = self.model[s_sim][a_sim]
    self.update_q_plan(s_sim, a_sim, r_sim, next_s_sim, alpha, gamma, k)
```

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:

(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Take action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Loop repeat n times:
    $S \leftarrow$ random previously observed state
    $A \leftarrow$ random action previously taken in S
    $R, S' \leftarrow Model(S,A)$
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

(e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$

```python
# Model Update
if state not in self.model: self.model[state] = {}
self.model[state][action] = (reward, next_state)

# Planning
for _ in range(num_plan):
    s_sim = random.choice(list(self.model.keys()))
    a_sim = random.choice(list(self.model[s_sim].keys()))
    r_sim, next_s_sim = self.model[s_sim][a_sim]
    self.update_q_plan(s_sim, a_sim, r_sim, next_s_sim, alpha, gamma, k)

state = next_state

if state in self.goals:
    self.num_steps.append(step + 1)
    break  # End planning if we hit a goal
if step == max_steps - 1:
    print("Warning: Max steps reached without hitting a goal.")
```

(e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$

```python
def update_q_plan(self, s, a, r, next_s, alpha, gamma, k):
    # Default planning: same as direct update
    self.update_q(s, a, r, next_s, alpha, gamma)
```

```python
def update_q(self, s, a, r, next_s, alpha, gamma):
    best_q = max(self.Q[next_s].values())
    self.Q[s][a] += alpha * (r + gamma * best_q - self.Q[s][a])
```

Dyna-Q+

Dyna-Q

- Too strict to old knowledge
- Hard to learn dynamic environment
- Lead to suboptimal policy

Encourage Bob to discover new possibilities

Dyna-Q+

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$

$$R \leftarrow R + \kappa\sqrt{t}$$

```python
class Variation1(Dyna_Q_Refined):
    def update_q_plan(self, s, a, r, next_s, alpha, gamma, k):
        r_plus = r + self.get_bonus(s, a, k)
        super().update_q(s, a, r_plus, next_s, alpha, gamma)
```

```python
def get_bonus(self, state, action, k):
    t = self.time_step - self.last_visit[(state, action)]
    return k * math.sqrt(t)
```

Dyna-Q+action

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon\text{-greedy}(S, Q)$

(c) Take action A; observe resultant reward, R, and state, S'

(d) $Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$

$$\mathrm{Q} \leftarrow Q + \kappa\sqrt{t}$$

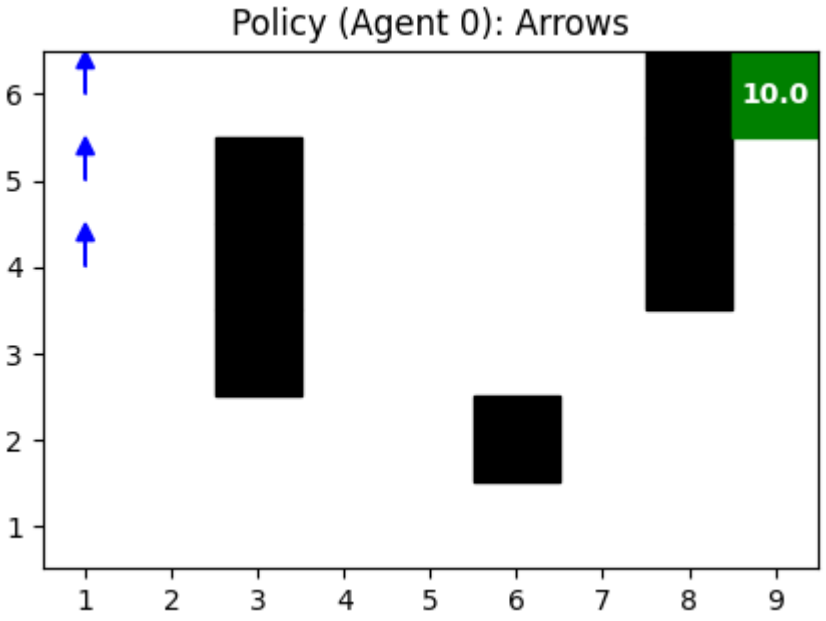
```
class Variation2(Dyna_Q_Refined):
    def get_action(self, state, epsilon, k=0):
        if random.random() < epsilon:
            return random.choice(['up', 'down', 'left', 'right'])
        # Selection uses Q + Bonus
        return max(self.Q[state], key=lambda a: self.Q[state][a] + self.get_bonus(state, a, k))
```

Scenario I

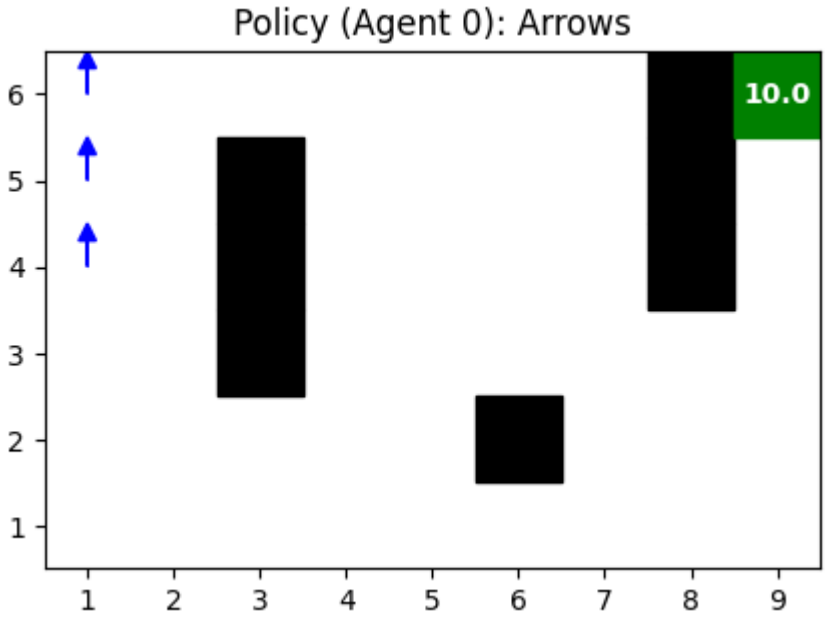
```
~~~~~~~~~~ Grid World ~~~~~~~~~~
 6 |  .    .    .    .    .    .    .    .    G  |
 5 |  .    .    .    .    .    .    .    .    .  |
 4 |  S    .    .    .    .    .    .    .    .  |
 3 |  .    .    .    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .  |
     ----------------------------------------
       1    2    3    4    5    6    7    8    9

--- Rewards ---
Goal at (9, 6): 10.00
```

Dyna-Q

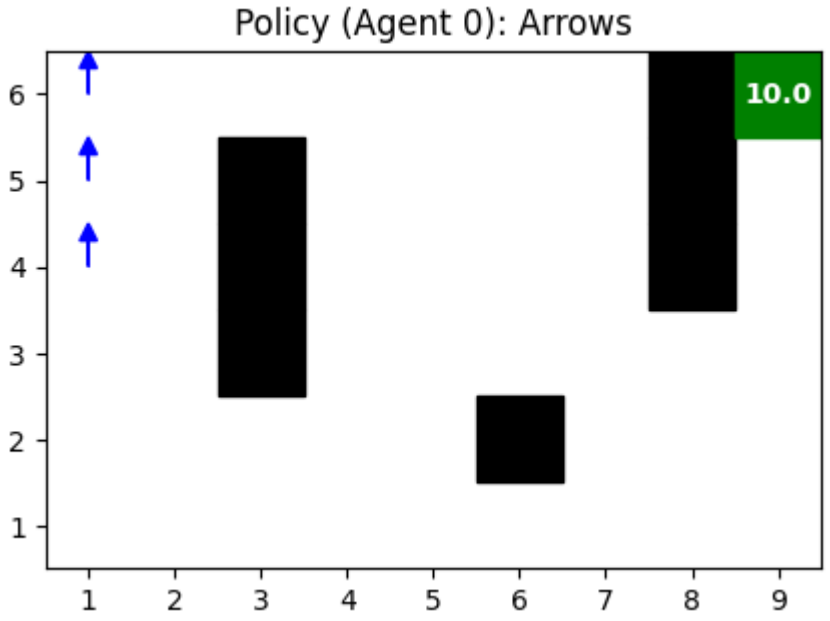
Ep 1



0 planning steps



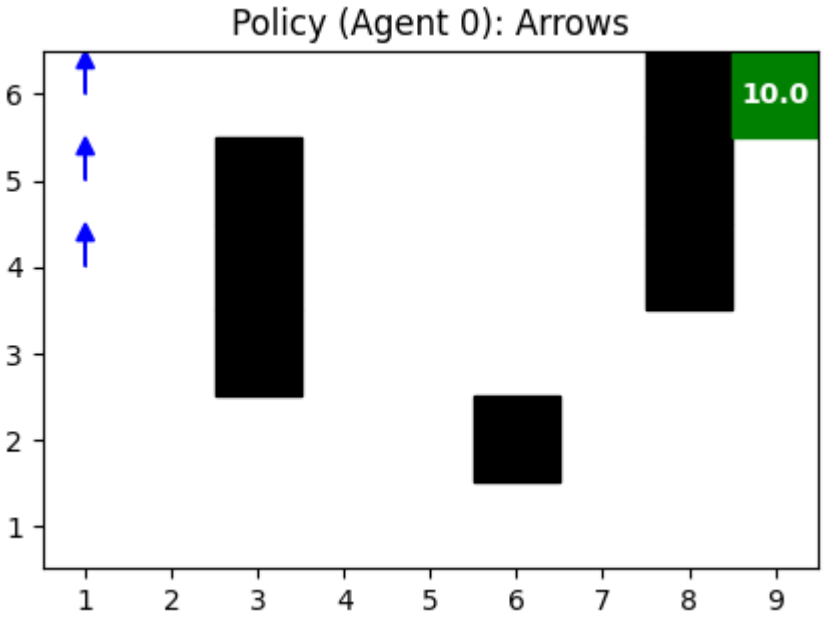
5 planning steps



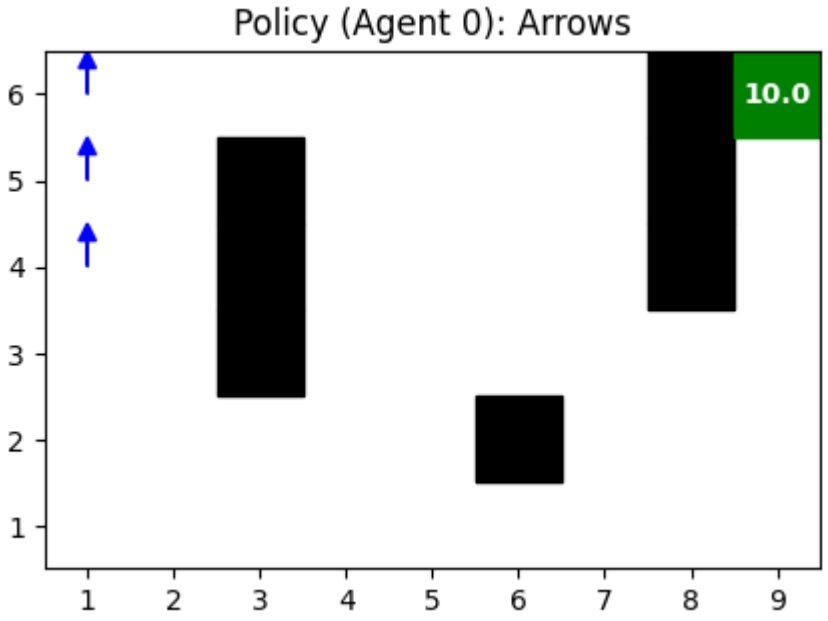
50 planning steps

Dyna-Q

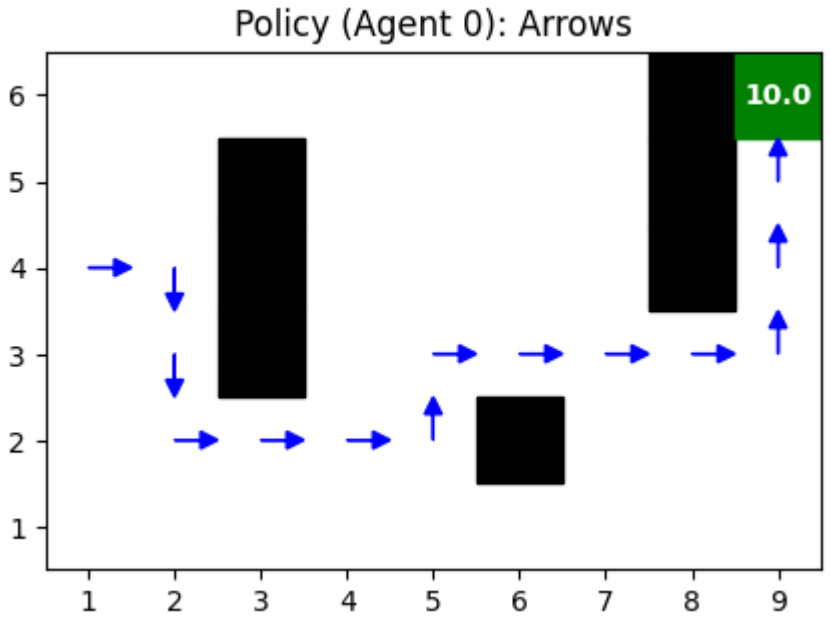
Ep 2



0 planning steps



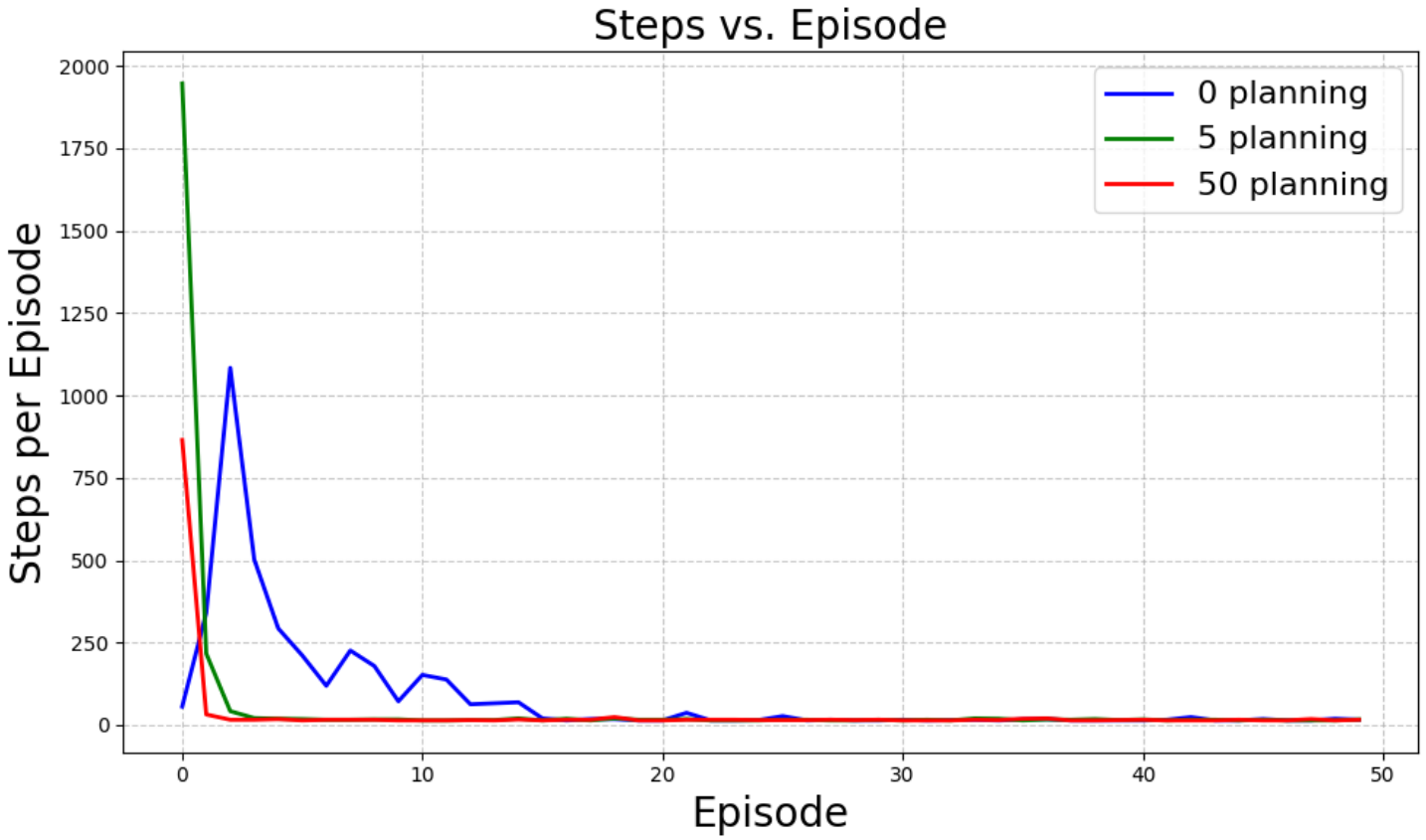
5 planning steps



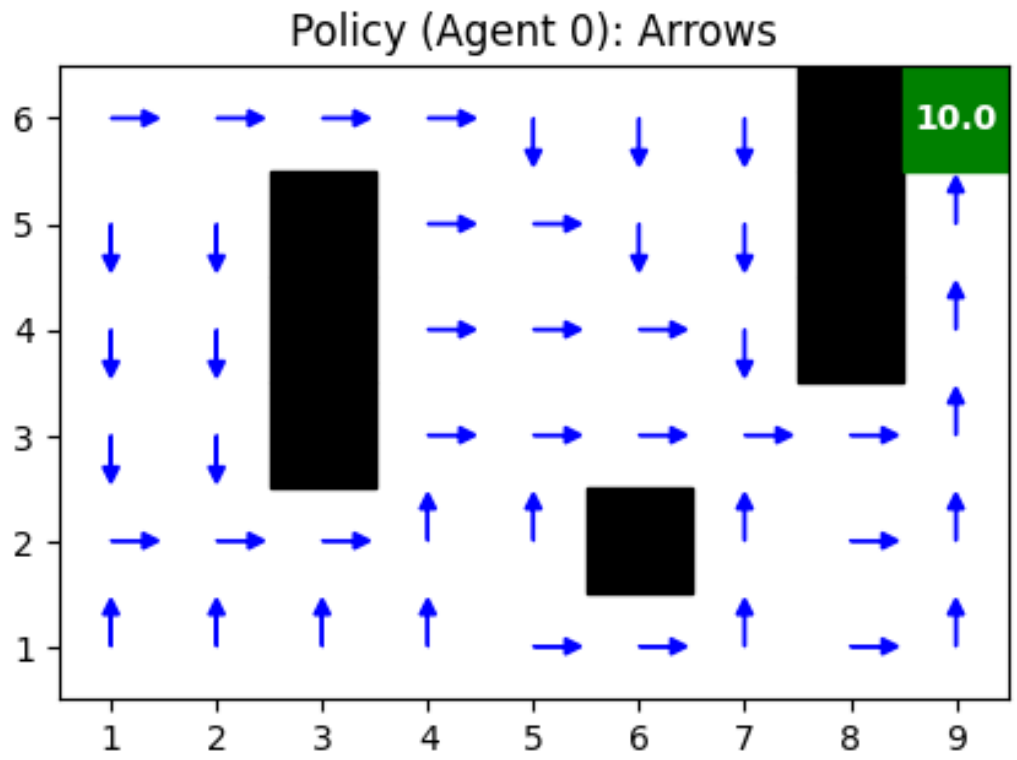
50 planning steps

Dyna-Q

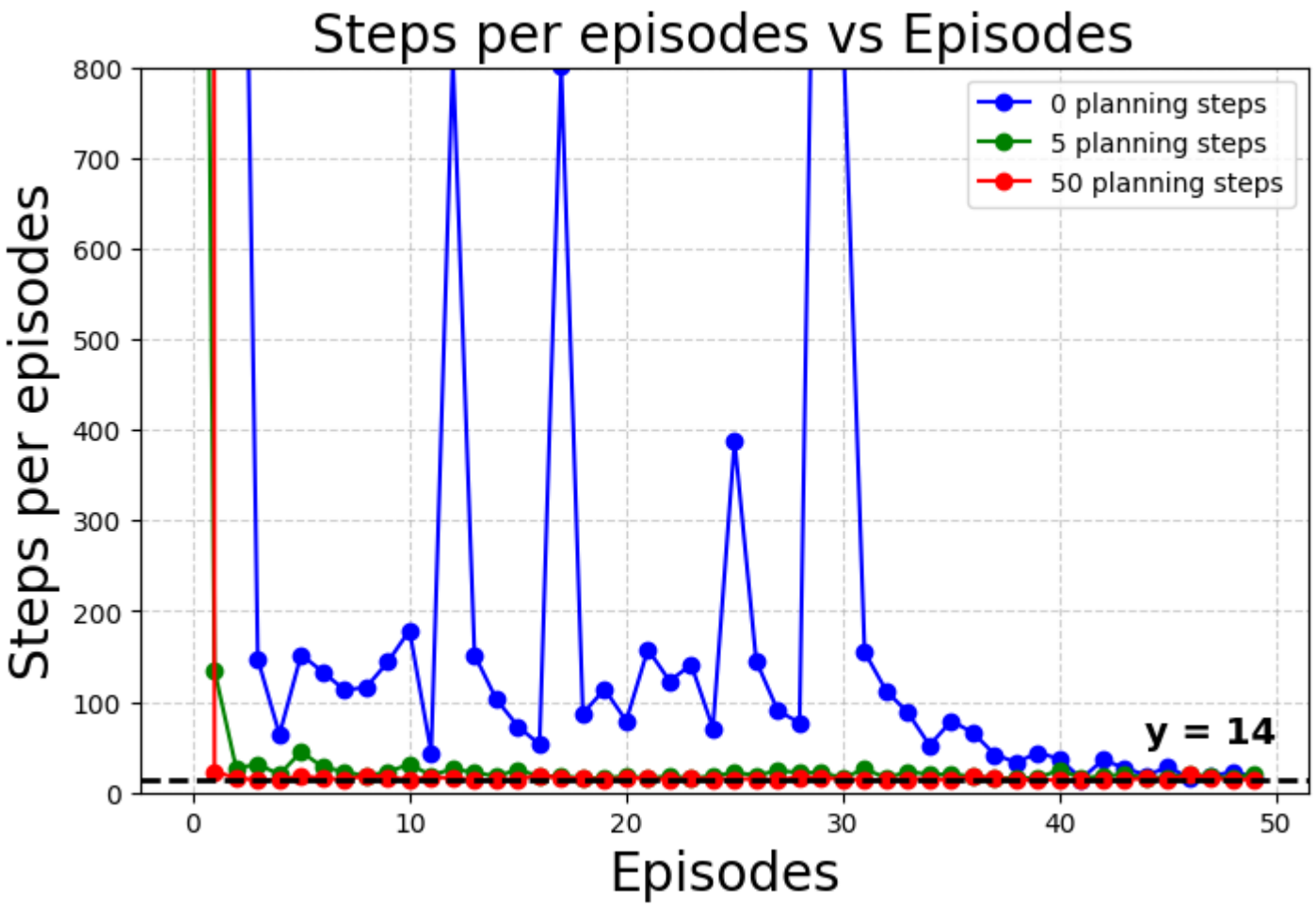
50 planning steps



Dyna-Q



50 planning steps

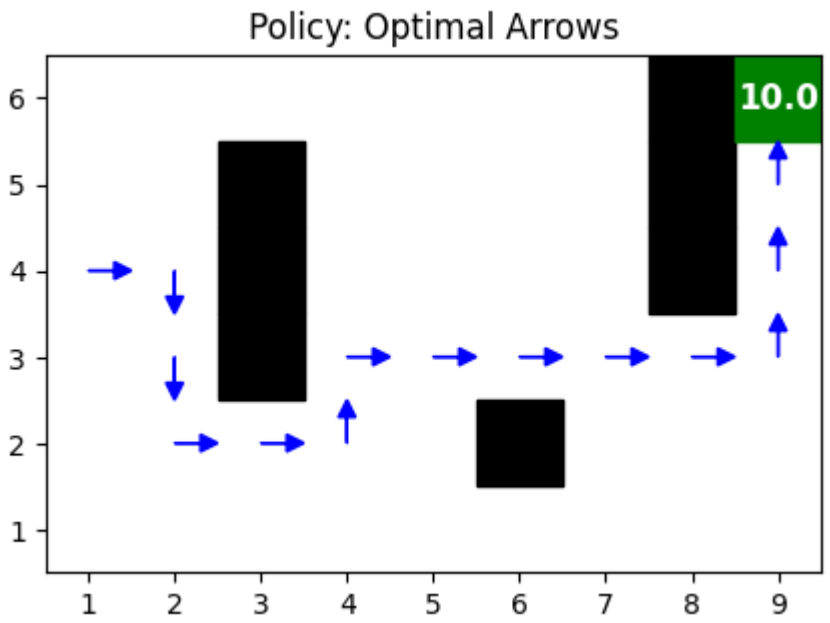


Scenario I

$$episode = 50, n = 50, \alpha = 0.1, \gamma = 0.95, \kappa = 0.0005$$

Dyna-Q

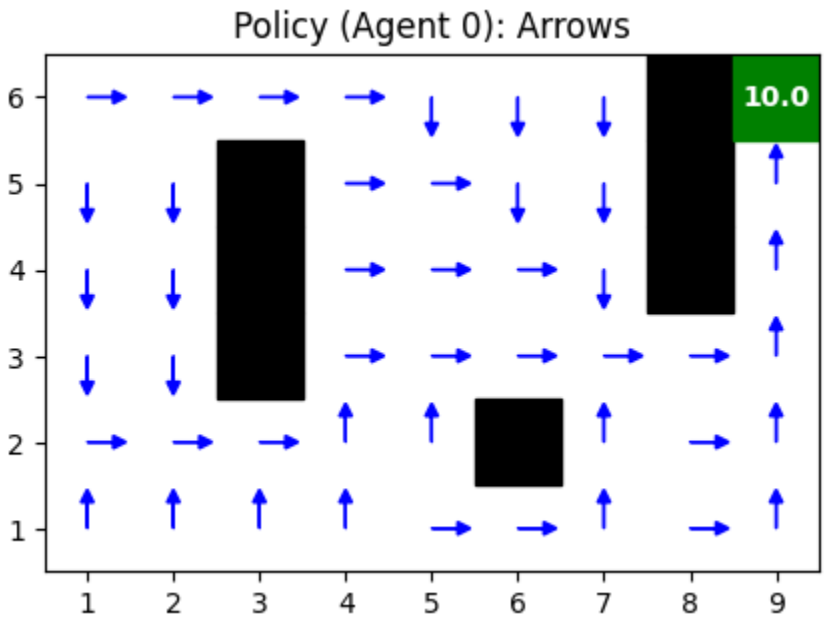
Dyna-Q+



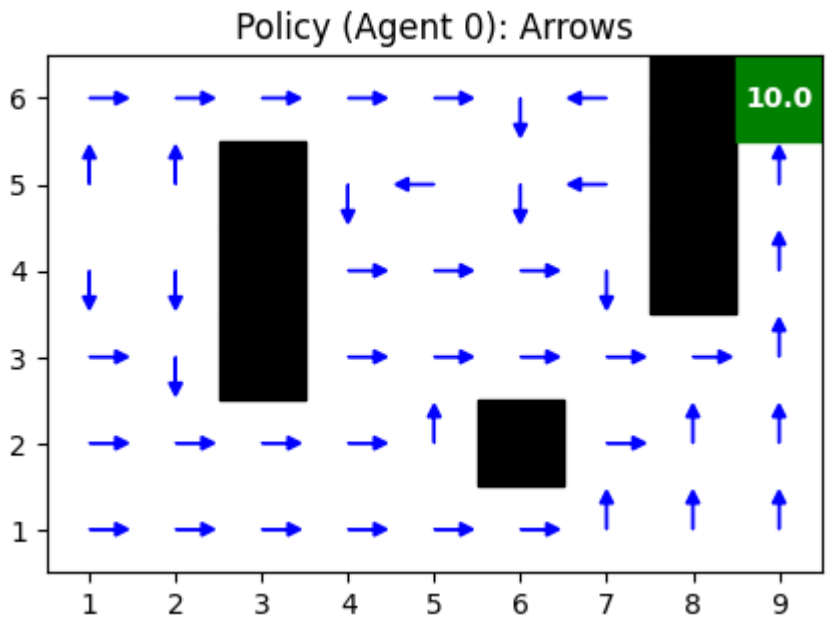
Dyna-Q+action

Scenario I

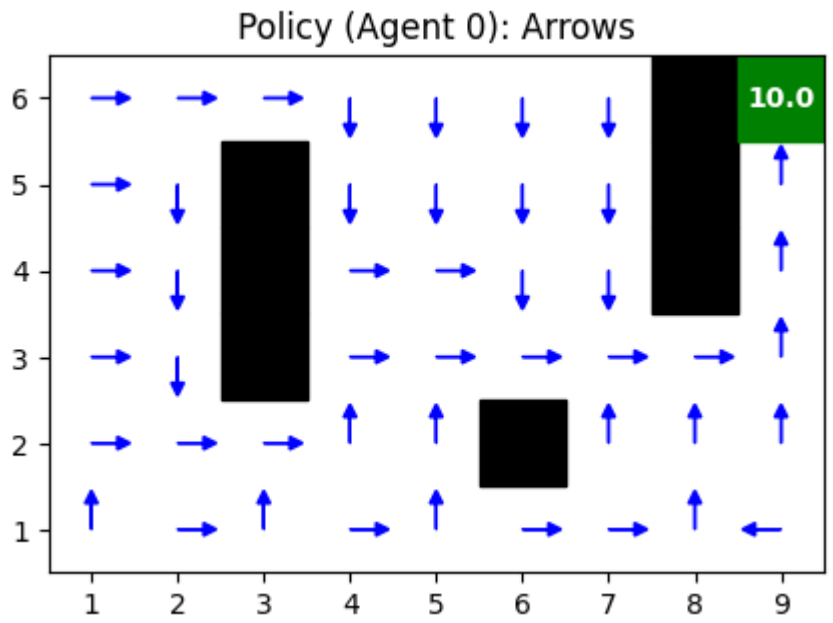
$$episode = 50, n = 50, \alpha = 0.1, \gamma = 0.95, \kappa = 0.0005$$



Dyna-Q



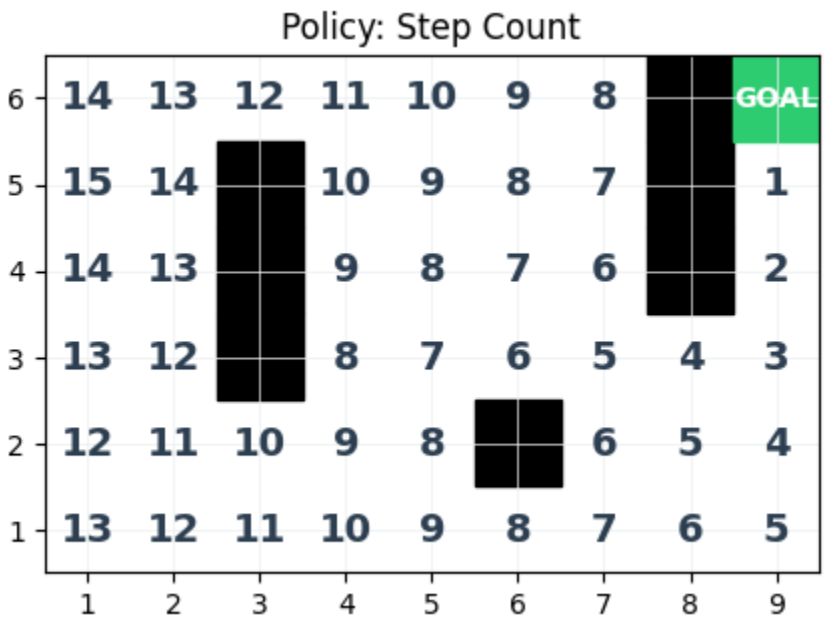
Dyna-Q+



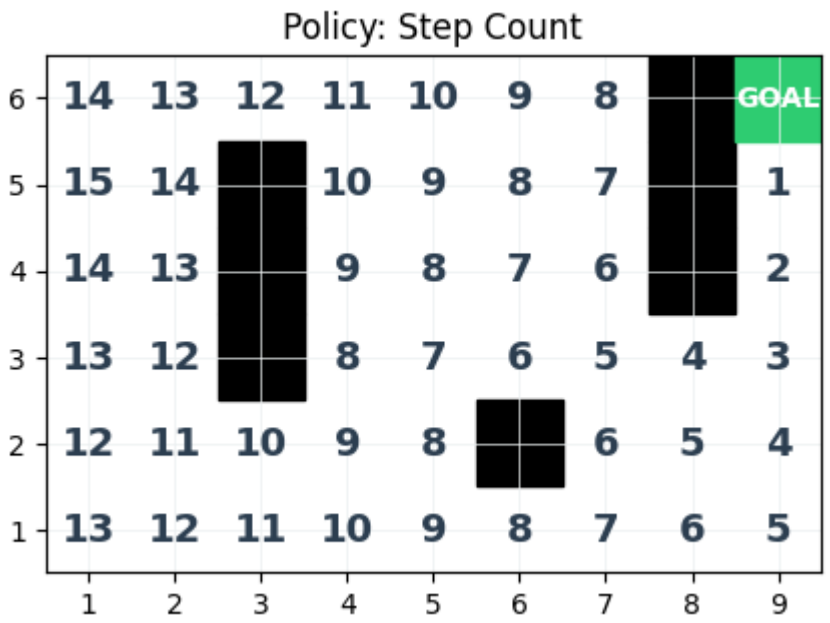
Dyna-Q+action

Scenario I

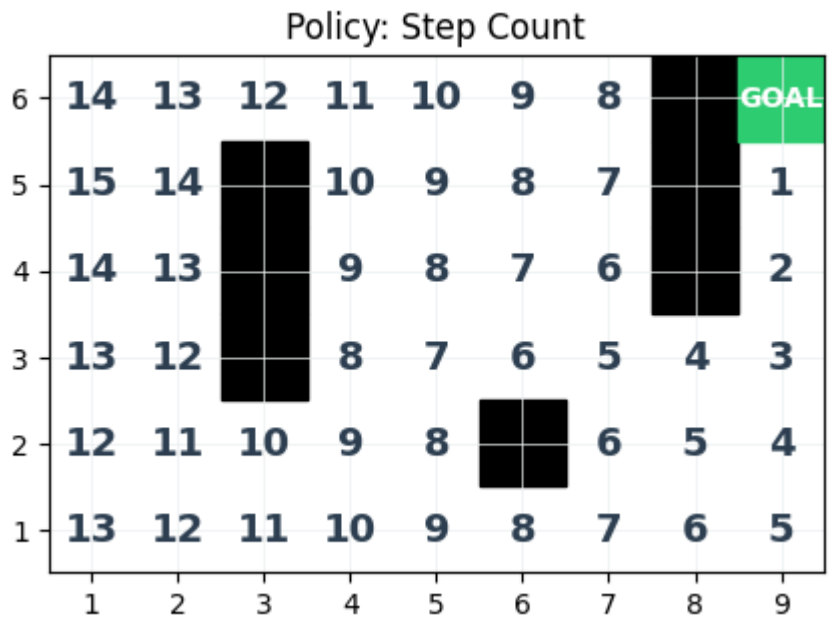
$$episode = 50, n = 50, \alpha = 0.1, \gamma = 0.95, \kappa = 0.0005$$



Dyna-Q



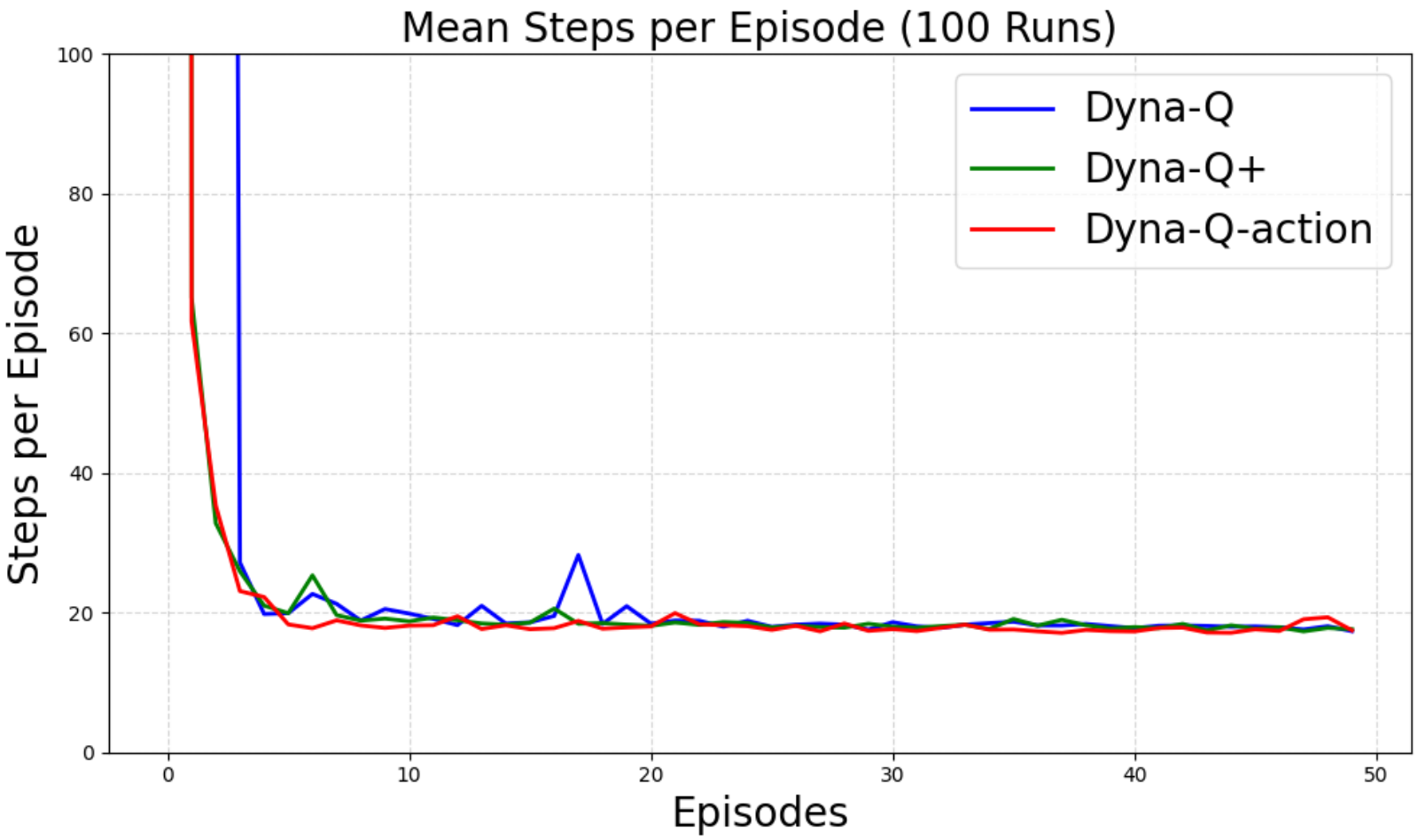
Dyna-Q+



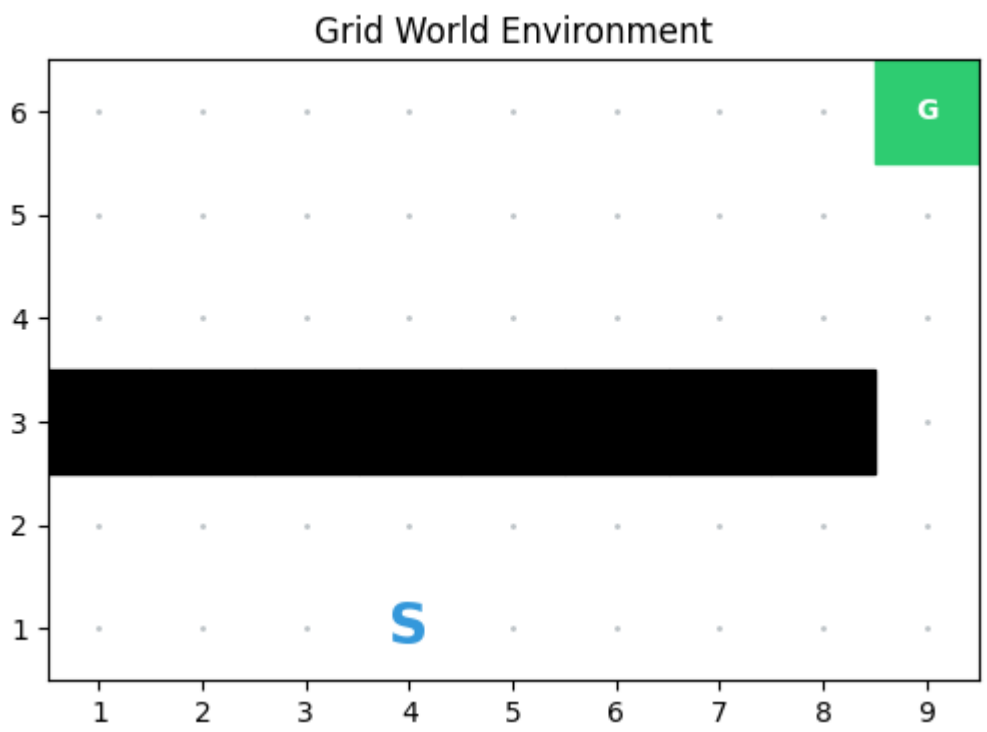
Dyna-Q+action

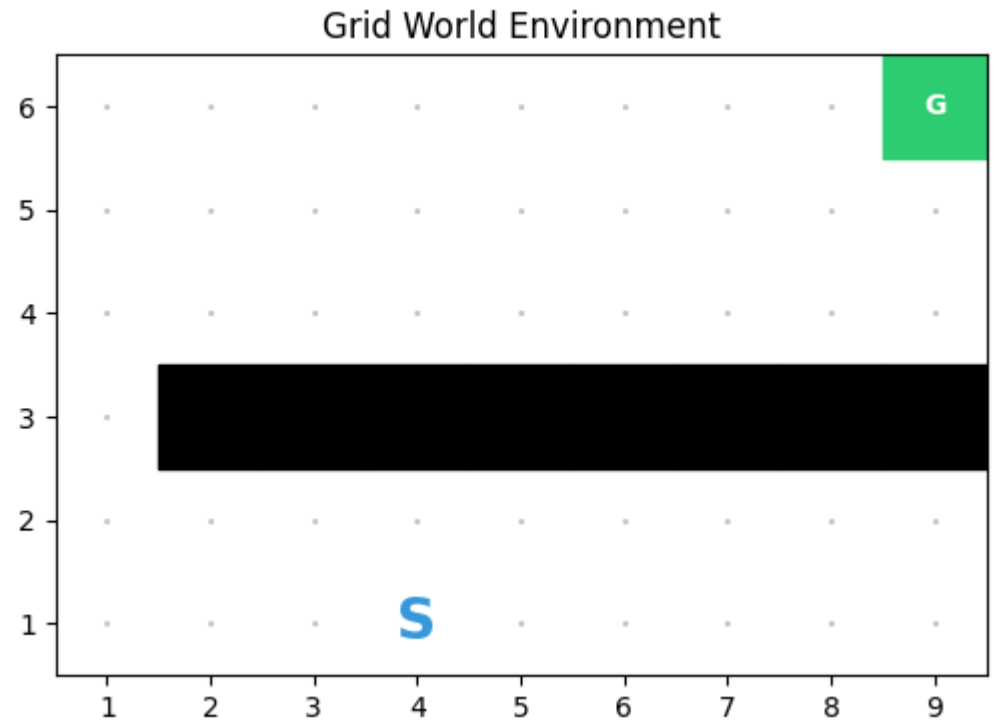
Scenario I

$$episode = 50, n = 50, \alpha = 0.1, \gamma = 0.95, \kappa = 0.03$$



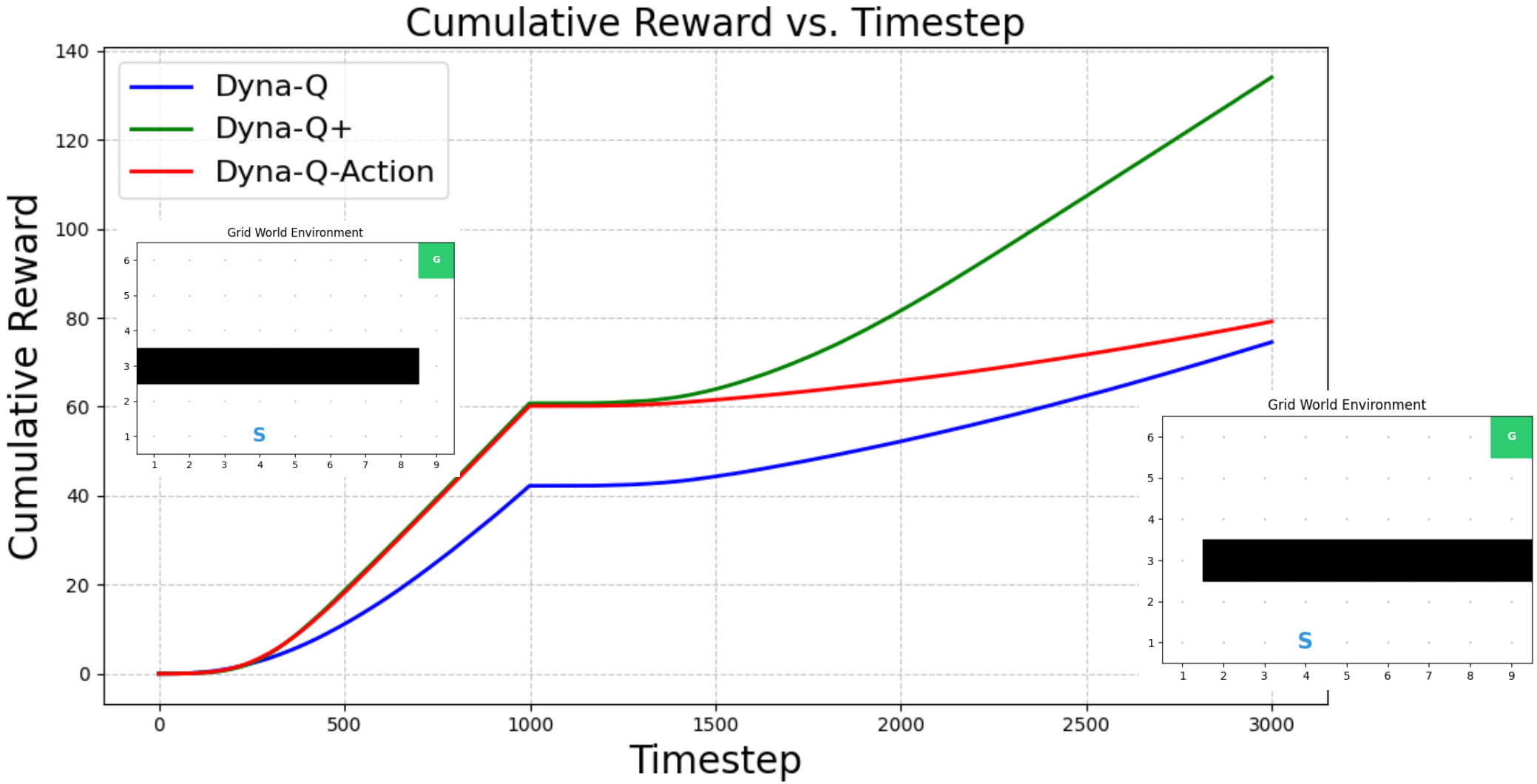
Scenario II



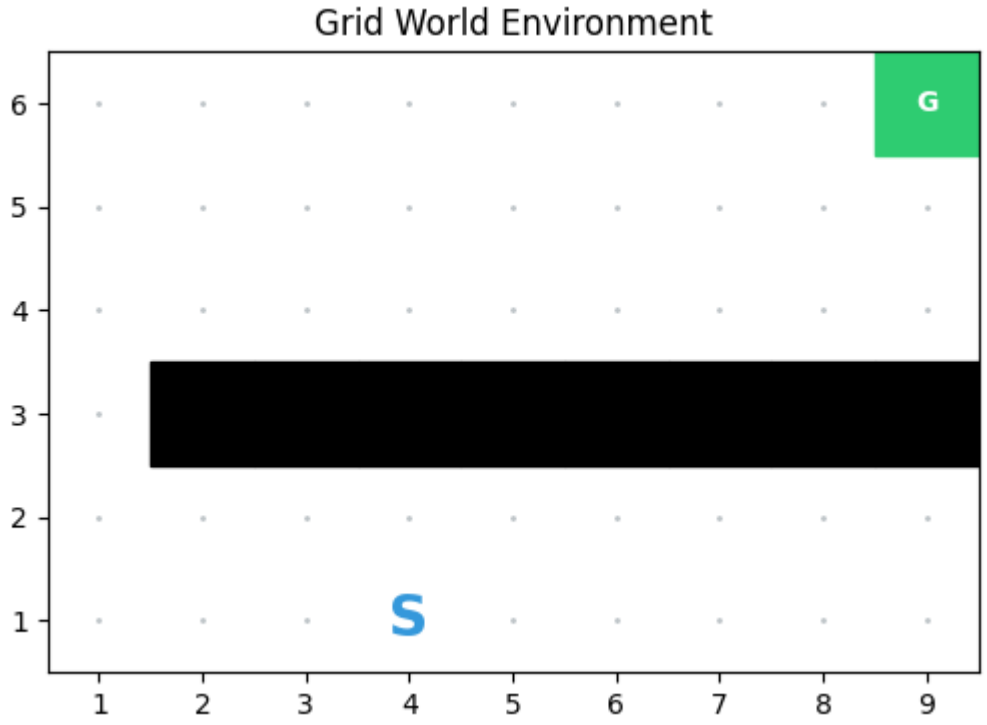


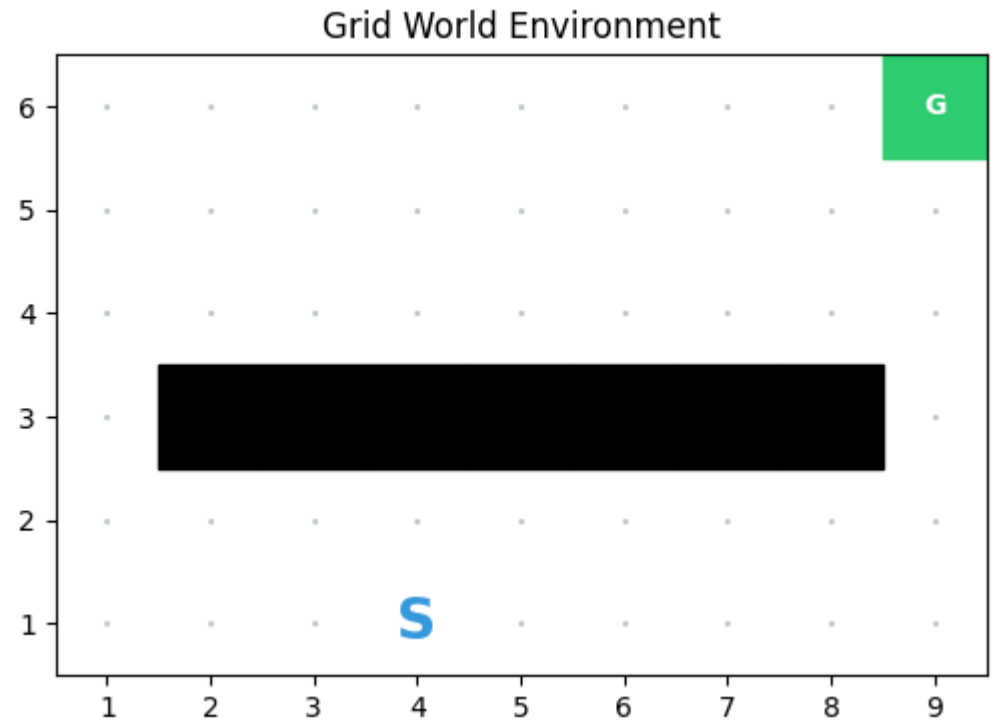
1000 time steps

Scenario II


Cumulative Reward vs. Timestep
Dyna-Q
Dyna-Q+
Dyna-Q-Action
Cumulative Reward
Timestep
0
20
40
60
80
100
120
140
0
500
1000
1500
2000
2500
3000
Grid World Environment
G
S
Grid World Environment
G
S

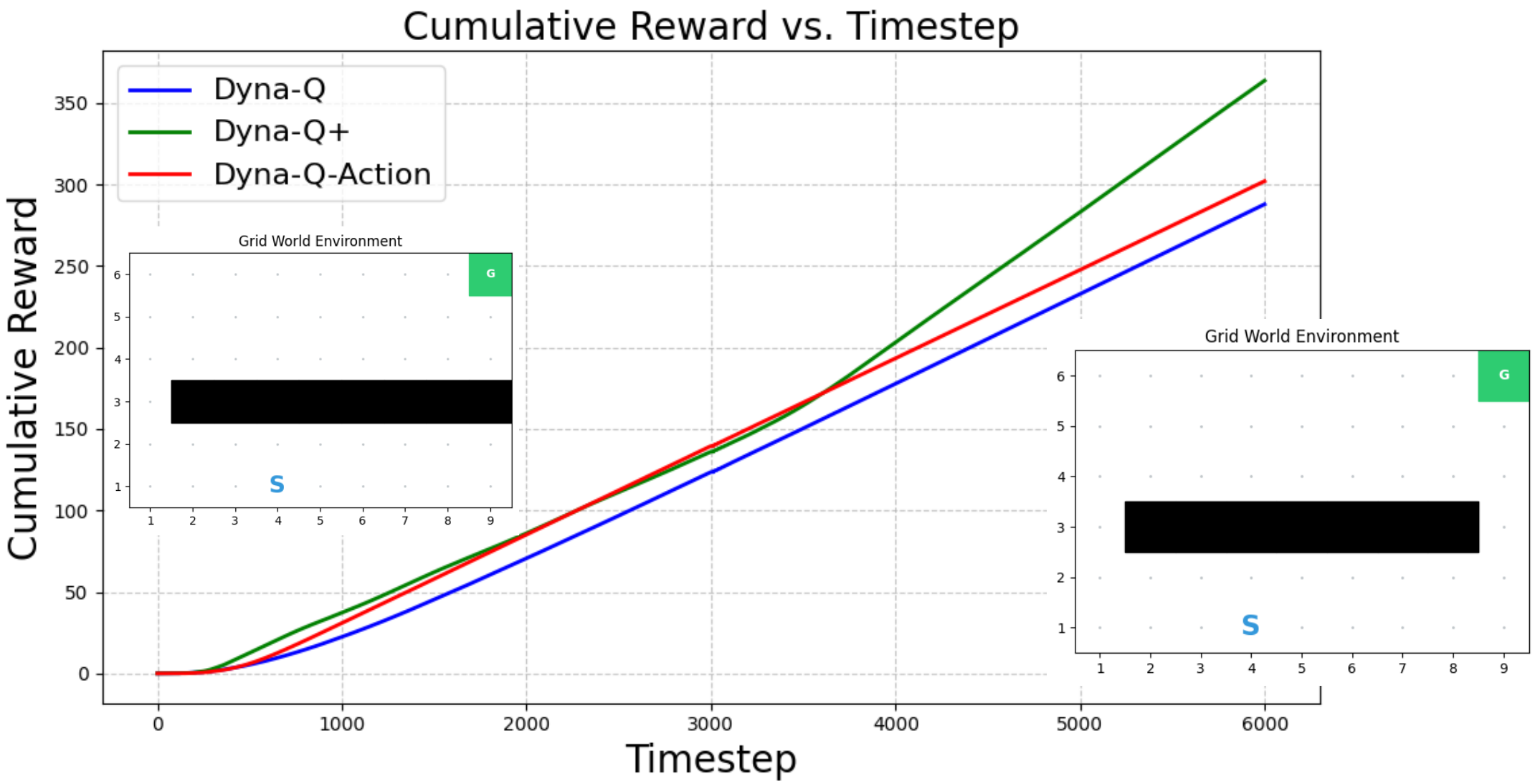
Scenario III



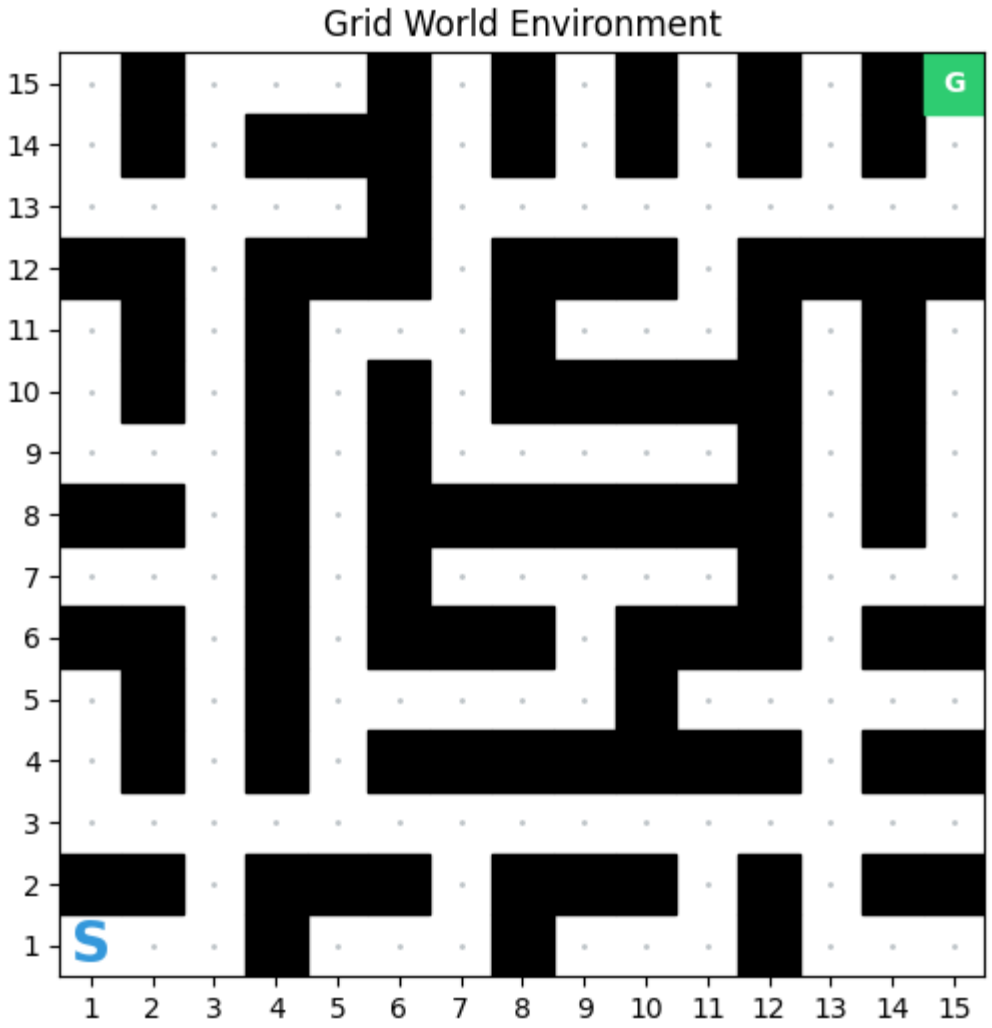


3000 time steps

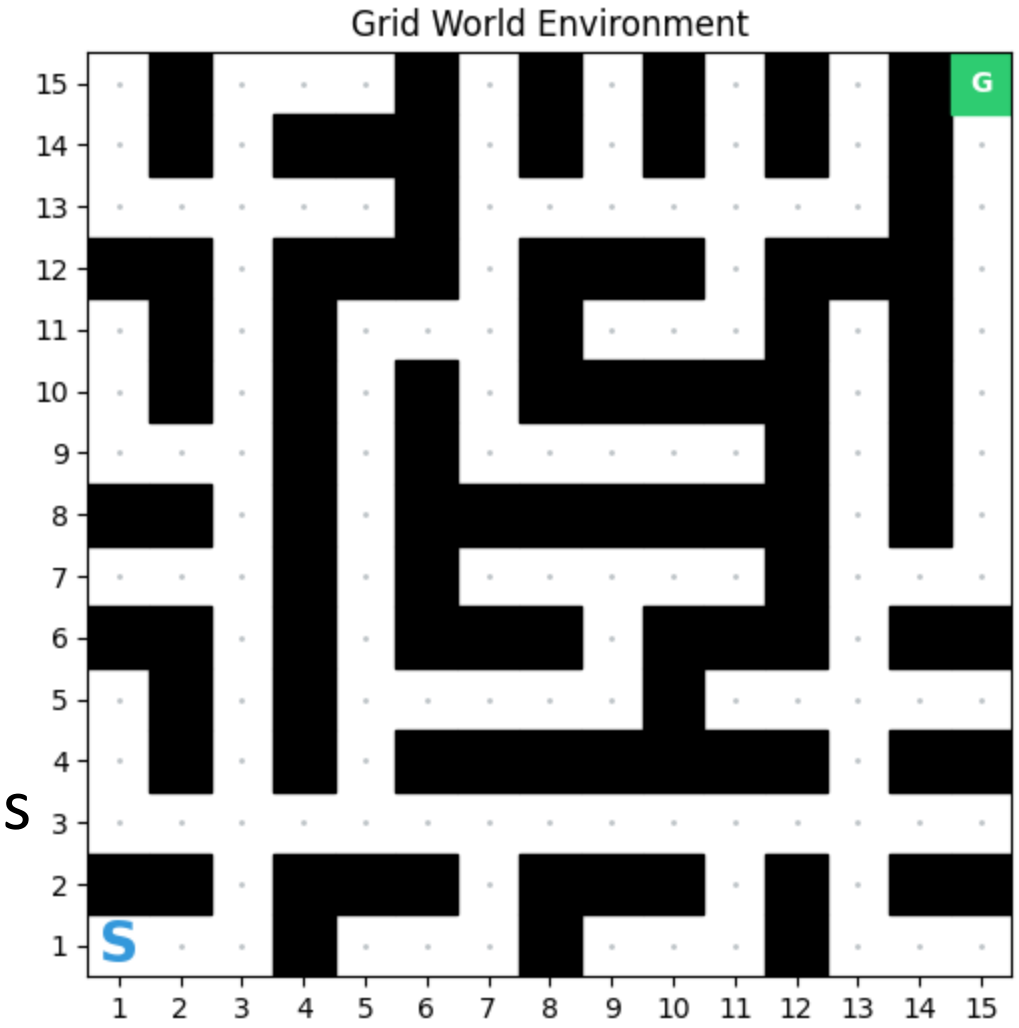
Scenario III



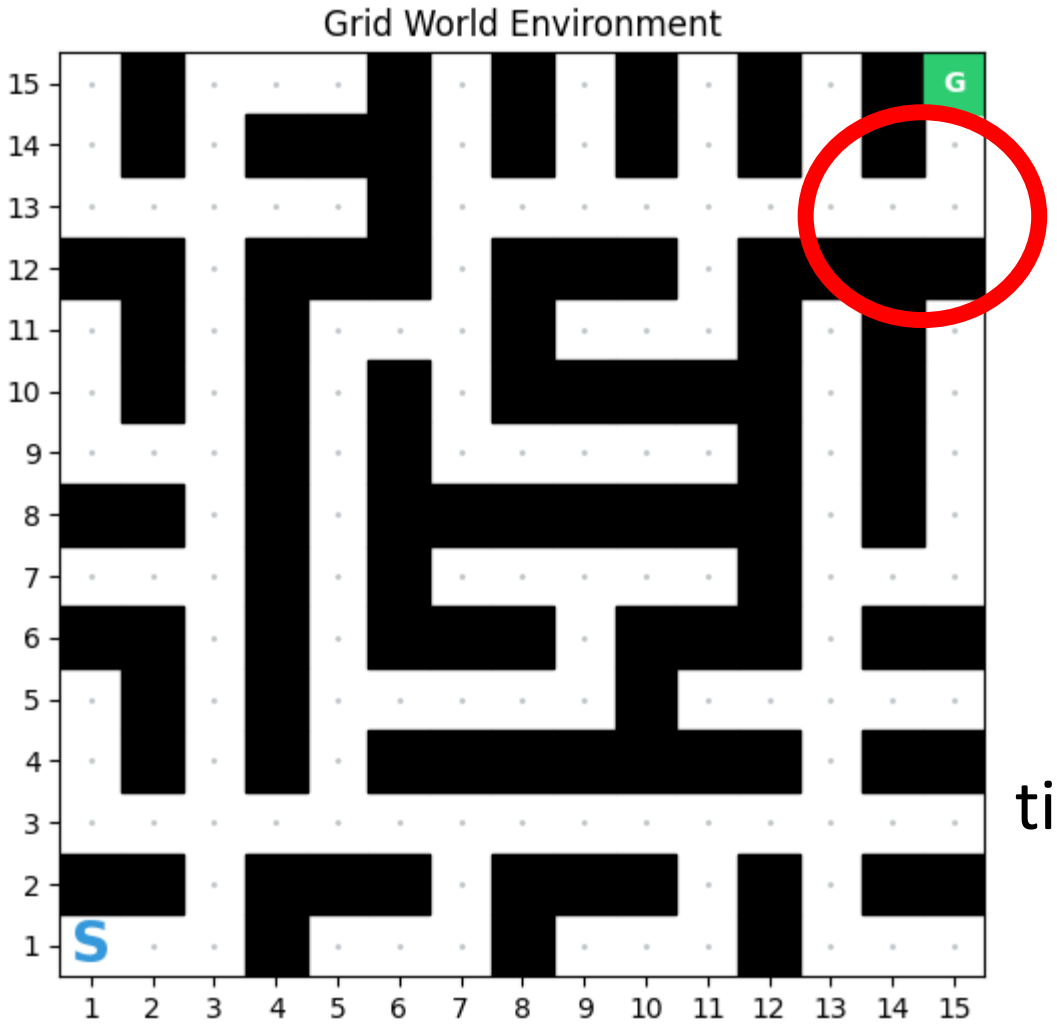
Scenario IV



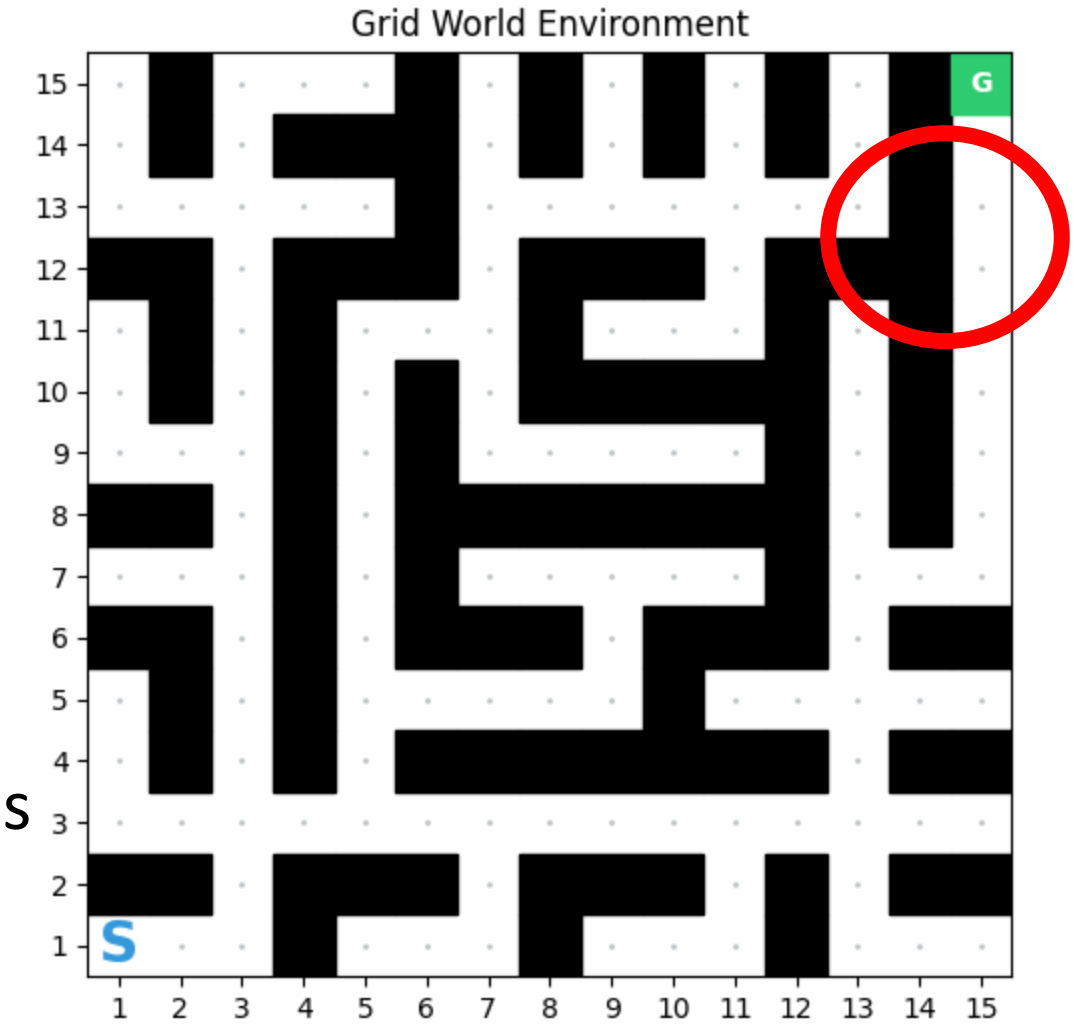
6000
time steps



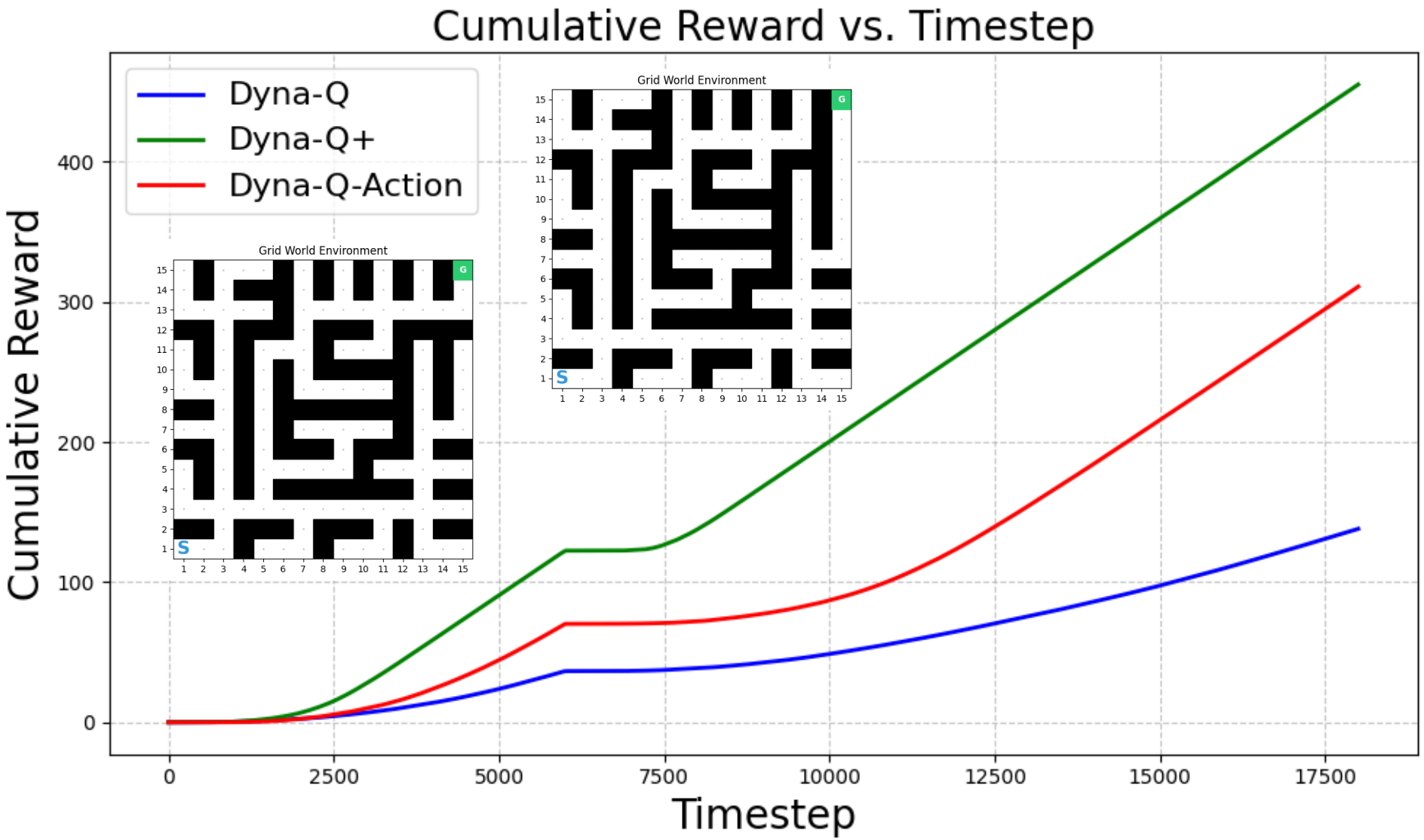
Scenario IV



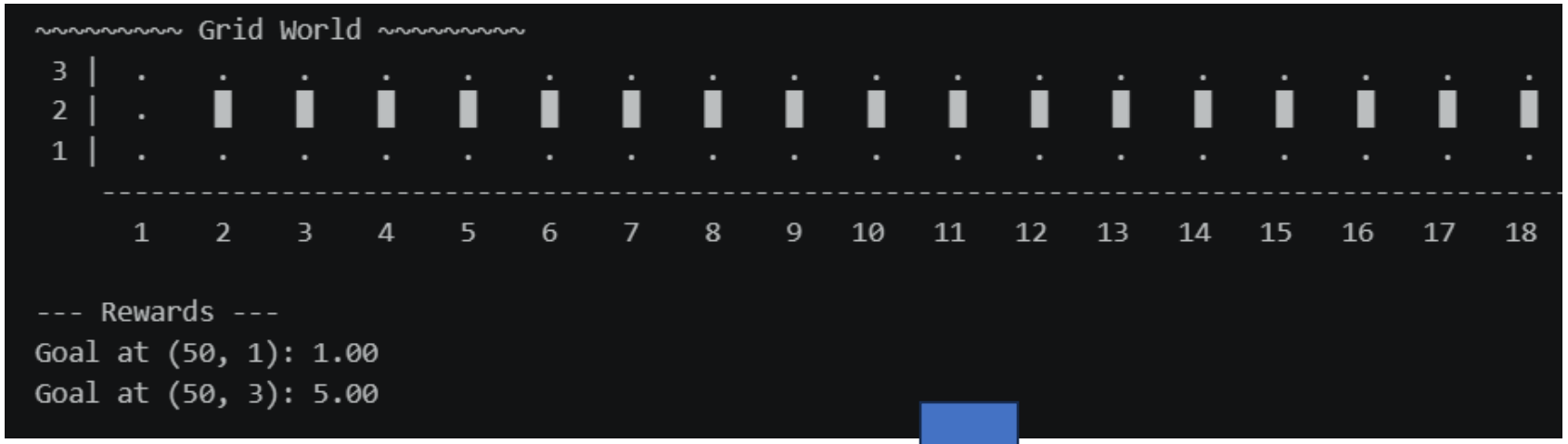
6000
time steps



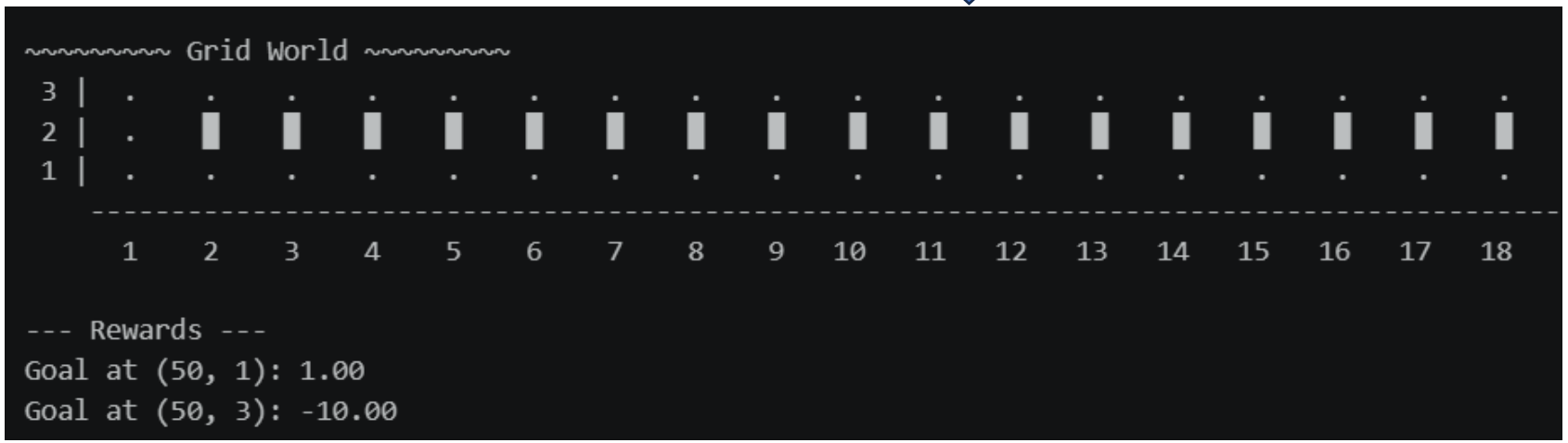
Scenario IV


Cumulative Reward vs. Timestep
Dyna-Q
Dyna-Q+
Dyna-Q-Action
Grid World Environment
Grid World Environment
Cumulative Reward
Timestep
0
100
200
300
400
0
2500
5000
7500
10000
12500
15000
17500

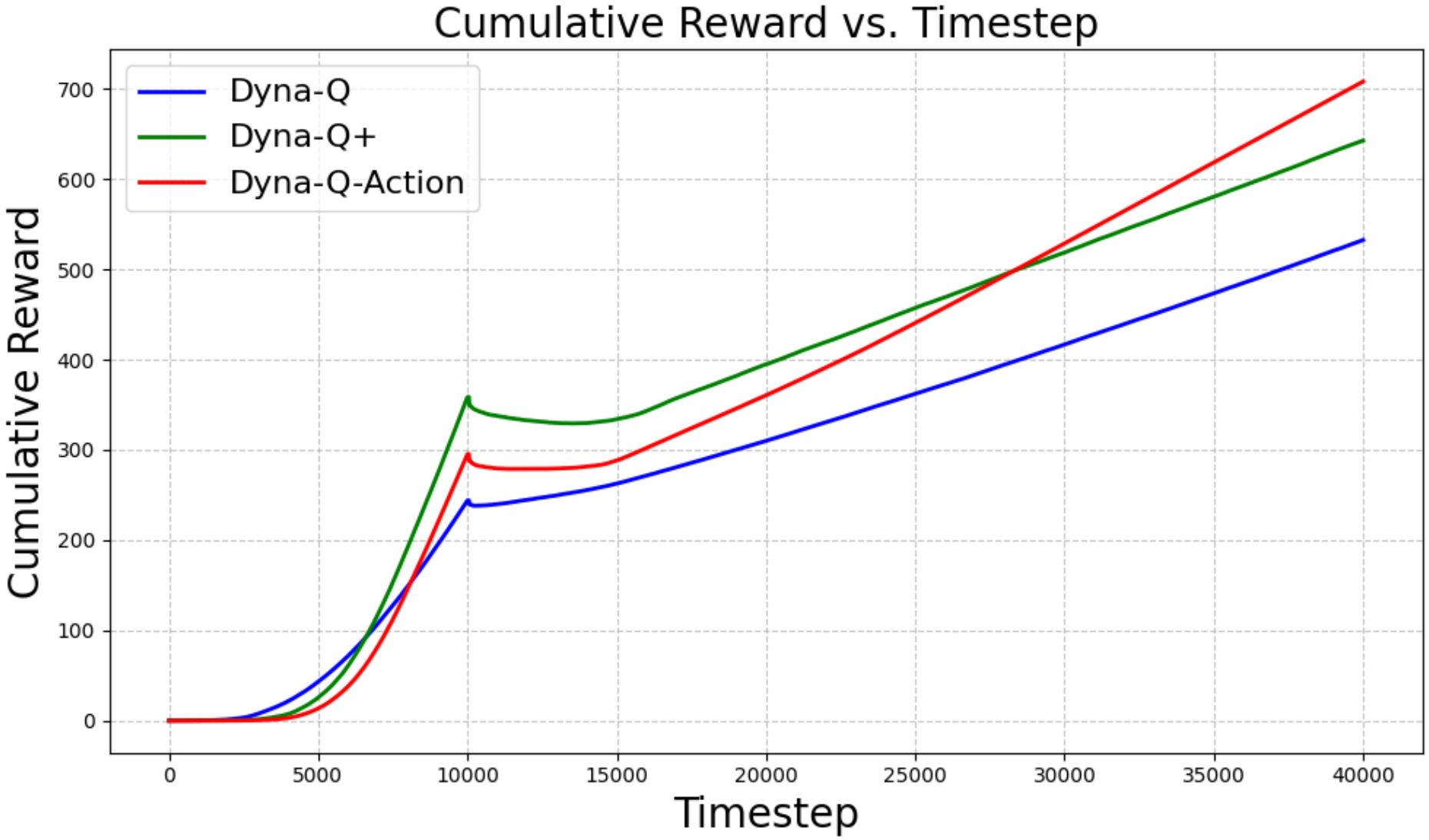
Scenario V



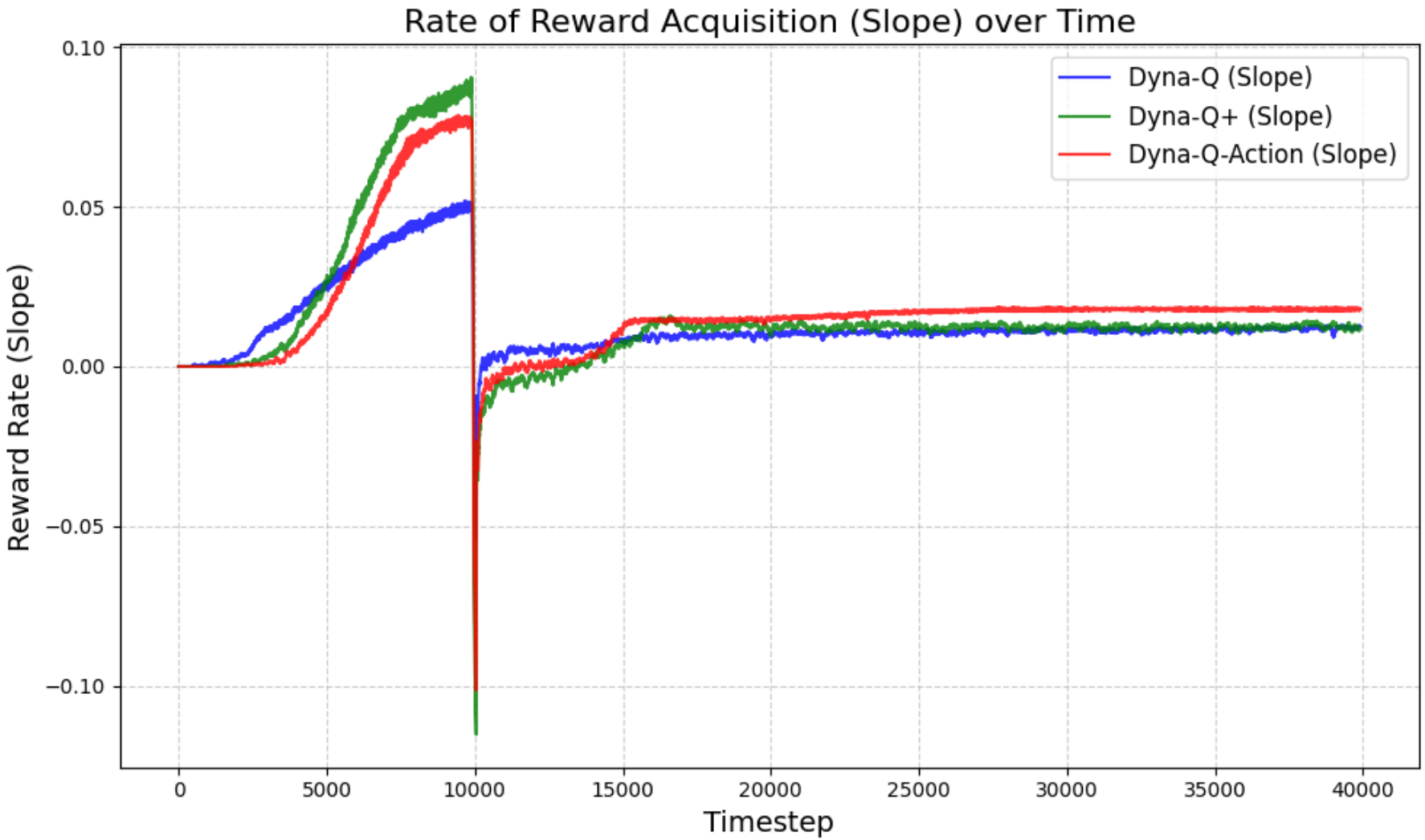
10000 time steps



Scenario V


Cumulative Reward vs. Timestep
Dyna-Q
Dyna-Q+
Dyna-Q-Action
Cumulative Reward
Timestep
0
100
200
300
400
500
600
700
0
5000
10000
15000
20000
25000
30000
35000
40000

Scenario V


Rate of Reward Acquisition (Slope) over Time
Dyna-Q (Slope)
Dyna-Q+ (Slope)
Dyna-Q-Action (Slope)
Reward Rate (Slope)
0.10
0.05
0.00
−0.05
−0.10
0
5000
10000
15000
20000
25000
30000
35000
40000
Timestep

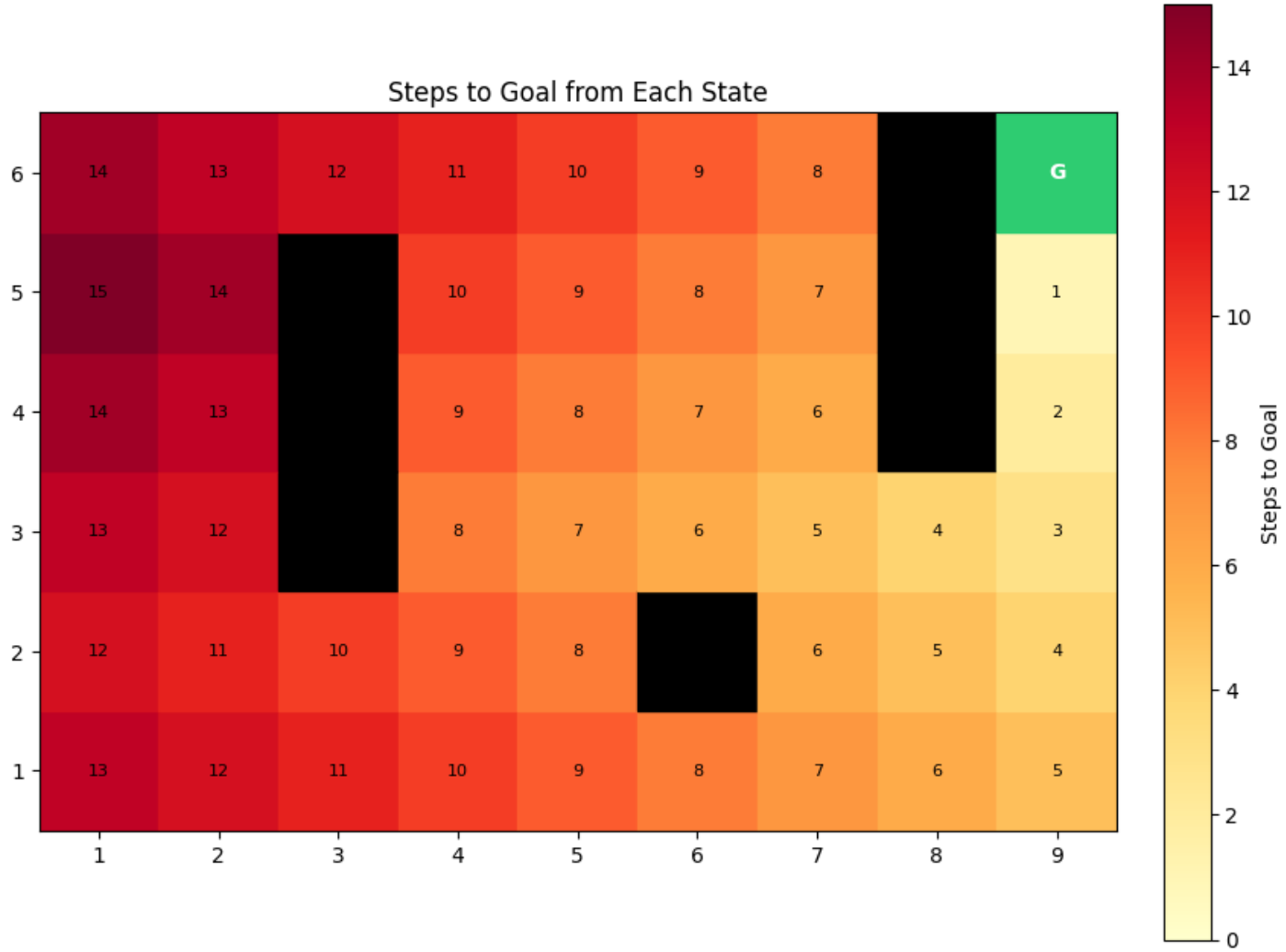
Conclusion

- The bonus is local. The agent only feels the "pull" of exploration if it is currently at state looking at action
- Often slower to react to changes
- Without the bonus in the updates, the information that an action hasn't been tried in a long time does not propagate back to previous states. In a gridworld, if a shortcut opens up ten steps away, an agent using the Exercise 8.4 approach won't "value" moving toward that area until it is physically standing next to the shortcut.
- It prevents the "optimism" from distorting the true value of the policy(unbiased estimation), which can be useful in stationary environments where you don't want the agent to keep re-checking "dead" paths forever.

Supporting slides

```
~~~~~~~~~ Grid World ~~~~~~~~~
 6 |  .    .    .    .    .    .    .    ▌    G  |
 5 |  .    .    ▌    .    .    .    .    ▌    .  |
 4 |  .    .    ▌    .    .    .    .    ▌    .  |
 3 |  .    .    ▌    .    .    .    .    .    .  |
 2 |  .    .    .    .    .    ▌    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .  |
    ------------------------------------------
      1    2    3    4    5    6    7    8    9

--- Rewards ---
Goal at (9, 6): 1.00
```

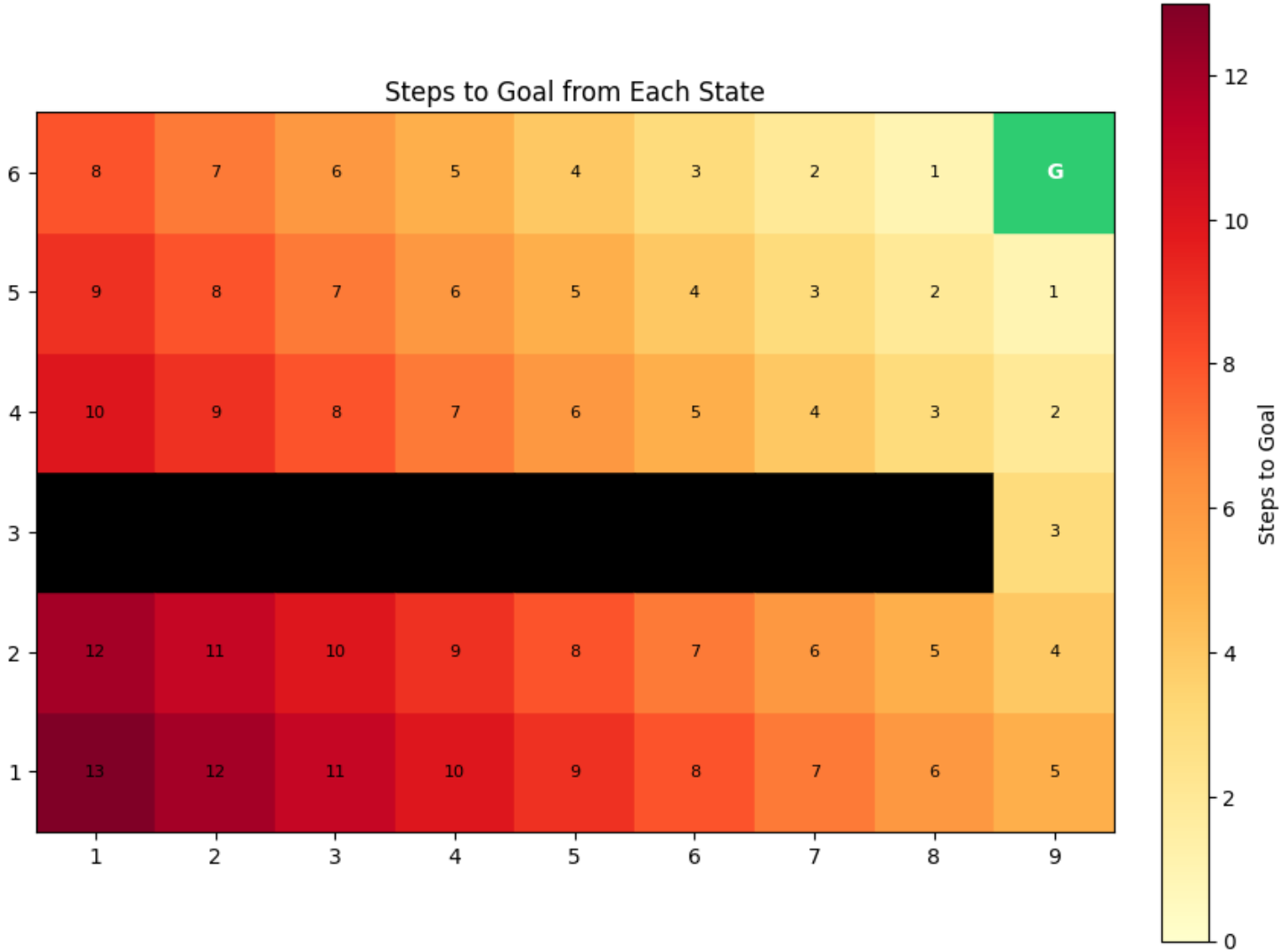
```
~~~~~~~ Optimal Policy ~~~~~~~
 6 | →  →  →  ↓  ↓  ↓  ↓  ▌  G  |
 5 | ↑  ↑  ▌  ↓  ↓  ↓  ↓  ▌  ↑  |
 4 | ↓  ↓  ▌  ↓  ↓  ↓  ↓  ▌  ↑  |
 3 | ↓  ↓  ▌  →  →  →  →  →  ↑  |
 2 | →  →  →  ↑  ↑  ▌  ↑  ↑  ↑  |
 1 | ↑  ↑  ↑  ↑  ↑  →  ↑  ↑  ↑  |
```



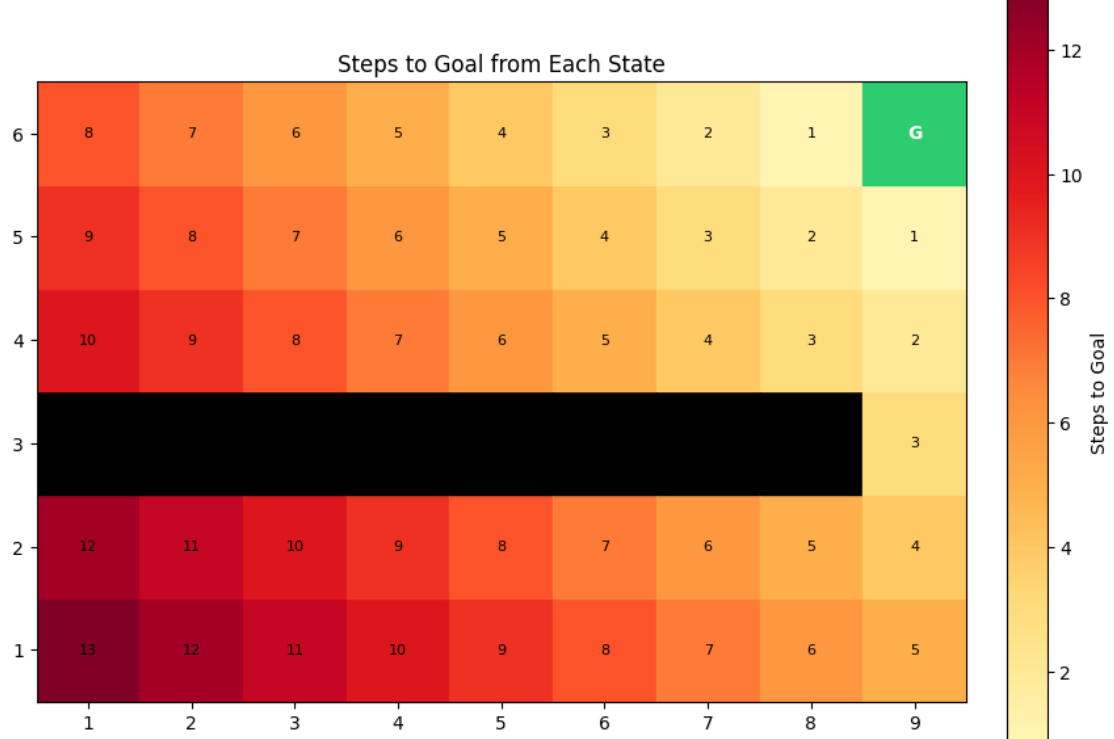
Supporting slides

```
~~~~~~~~~ Grid World ~~~~~~~~~
 6 |  .    .    .    .    .    .    .    .    G  |
 5 |  .    .    .    .    .    .    .    .    .  |
 4 |  .    .    .    .    .    .    .    .    .  |
 3 |  ▮    ▮    ▮    ▮    ▮    ▮    ▮    ▮    .  |
 2 |  .    .    .    .    .    .    .    .    .  |
 1 |  .    .    .    .    .    .    .    .    .  |
    -------------------------------------------
      1    2    3    4    5    6    7    8    9

--- Rewards ---
Goal at (9, 6): 1.00
```

```
~~~~~~~~ Optimal Policy ~~~~~~~~
 6 | →  →  →  →  →  →  →  →  G  |
 5 | ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  |
 4 | ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  |
 3 | ▮  ▮  ▮  ▮  ▮  ▮  ▮  ▮  ↑  |
 2 | →  →  →  →  →  →  →  →  ↑  |
 1 | ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  |
```



Policy (Agent 0): Steps
GOAL


Steps to Goal from Each State
G
Steps to Goal

Policy (Agent 0): Steps
GOAL

Policy (Agent 0): Steps
GOAL